Indian Institute of Information Technology, Allahabad

Convolutional Neural Network

By

Dr. Shiv Ram Dubey
Assistant Professor
Computer Vision And Biometrics Lab (CVBL)
Department Of Information Technology
Indian Institute Of Information Technology, Allahabad

Email: srdubey@iiita.ac.in Web: https://profile.iiita.ac.in/srdubey/

DISCLAIMER

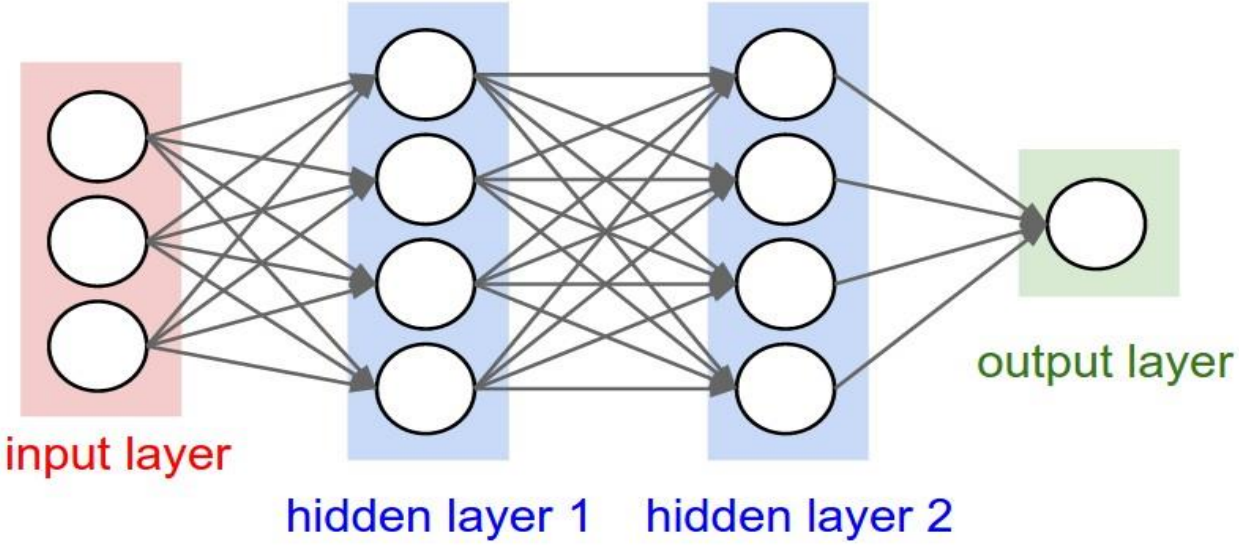
The content (text, image, and graphics) used in this slide are adopted from many sources for Academic purposes. Broadly, the sources have been given due credit appropriately. However, there is a chance of missing out some original primary sources. The authors of this material do not claim any copyright of such material.

OUTLINE

- Convolutional Neural Networks (CNNs):
  - Architecture
  - Different layers
  - Progress
- CNN Architectures
  - AlexNet
  - VGG
  - GoogleNet
  - ResNet

Most material from CS231n CNN for Visual Recognition course at Stanford

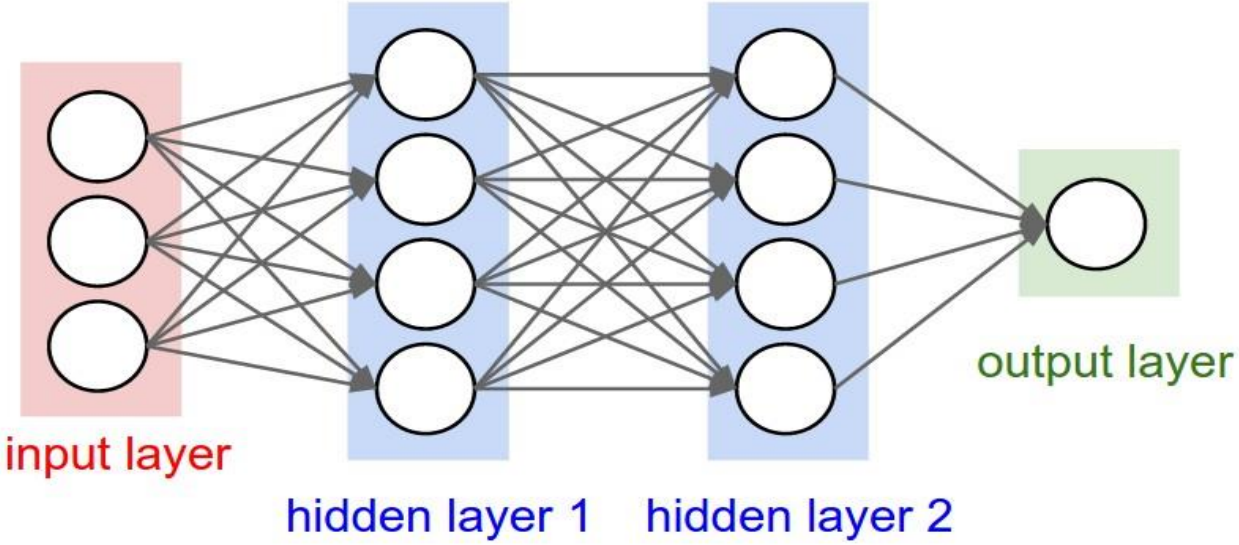
MULTI-LAYER NEURAL NETWORK & IMAGE



How to apply NN over Image?

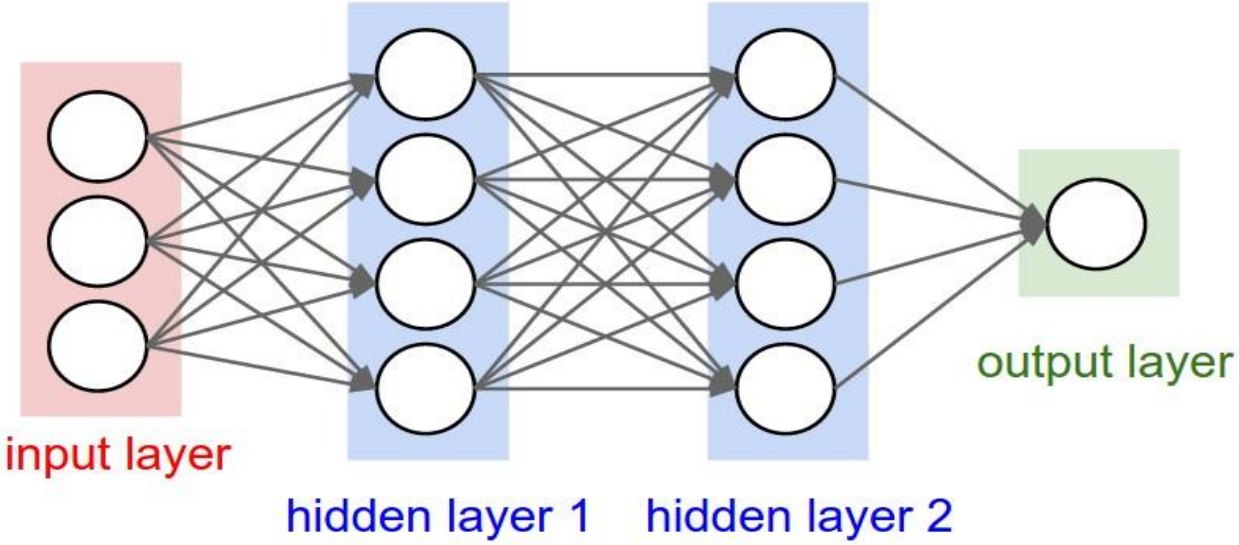
MULTI-LAYER NEURAL NETWORK & IMAGE

Stretch pixels in single column vector



MULTI-LAYER NEURAL NETWORK & IMAGE

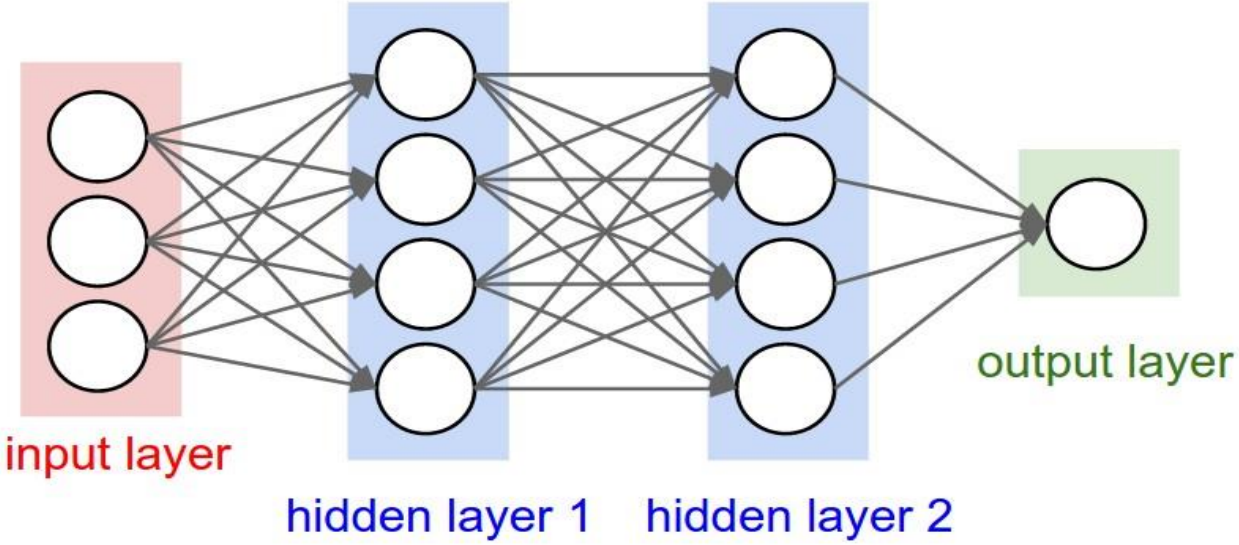
Stretch pixels in single column vector



Problems ?

MULTI-LAYER NEURAL NETWORK & IMAGE

Stretch pixels in single column vector



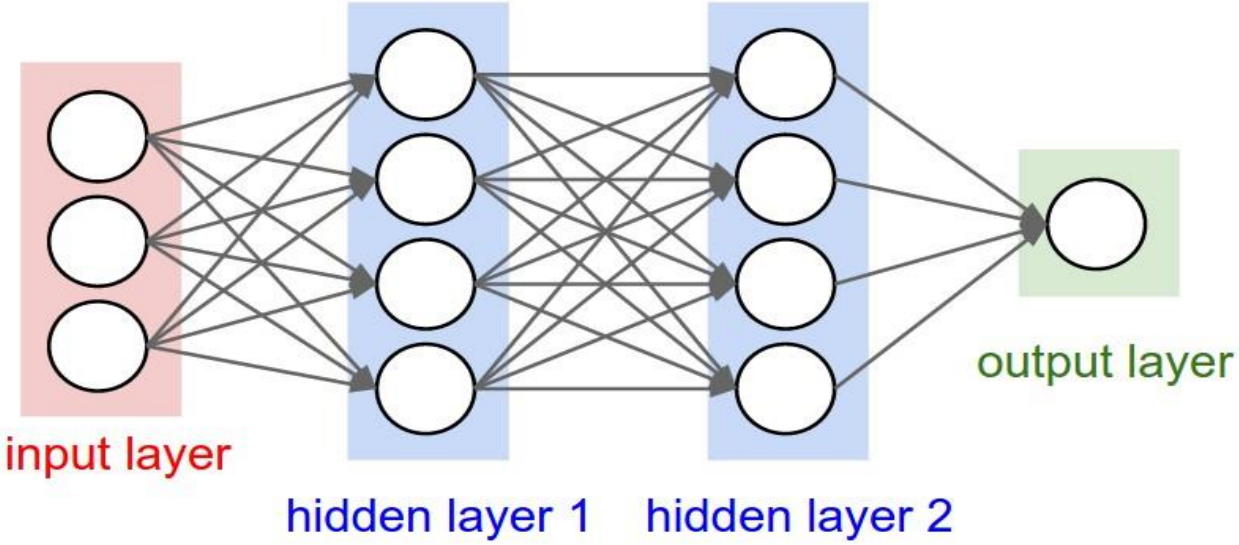
Problems:

High dimensionality

Local relationship

MULTI-LAYER NEURAL NETWORK & IMAGE

Stretch pixels in single column vector



Problems:

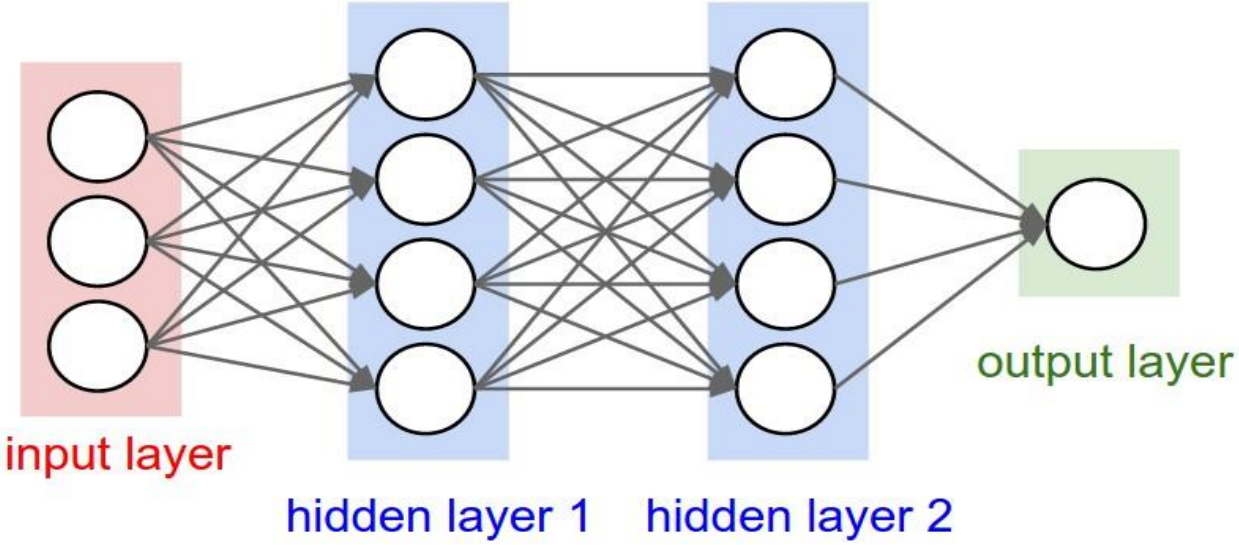
High dimensionality

Local relationship

Solution ?

MULTI-LAYER NEURAL NETWORK & IMAGE

Stretch pixels in single column vector



Problems:

High dimensionality

Local relationship

Solution:

Convolutional Neural Network

CONVOLUTIONAL NEURAL NETWORKS

- Also known as

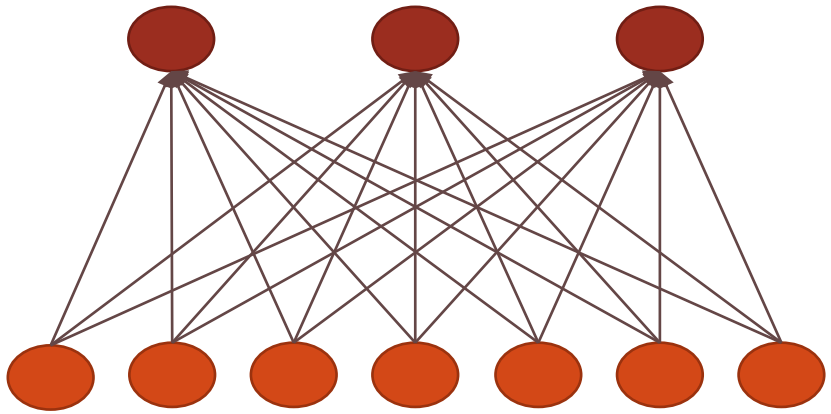
  CNN,

  ConvNet,

  DCN

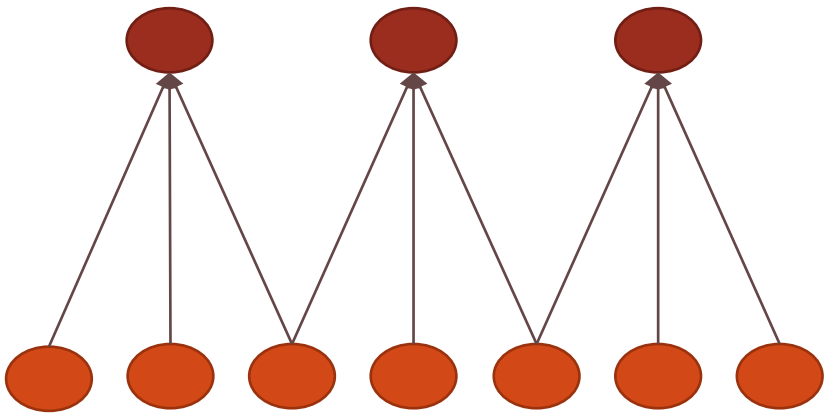
- CNN = a multi-layer neural network with
  1. Local connectivity
  2. Weight sharing

CNN: LOCAL CONNECTIVITY



Hidden layer

Input layer

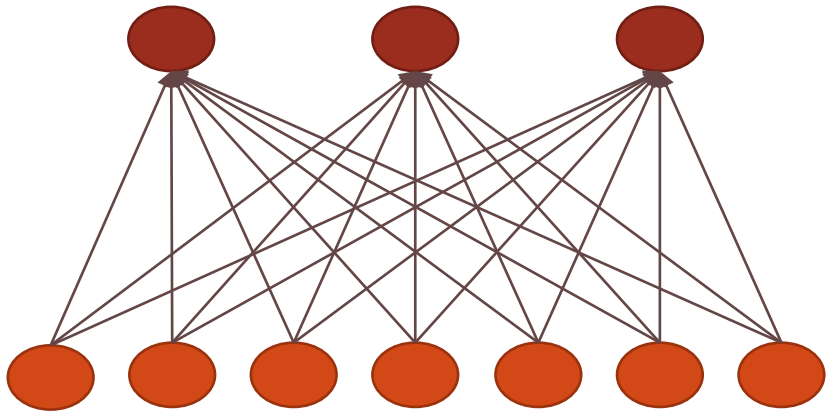


Global connectivity

- # input units (neurons): 7
- # hidden units: 3

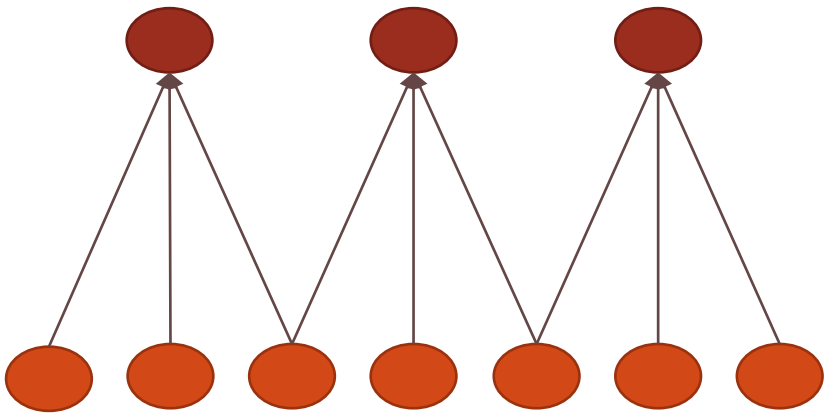
Local connectivity

CNN: LOCAL CONNECTIVITY



Hidden layer

Input layer

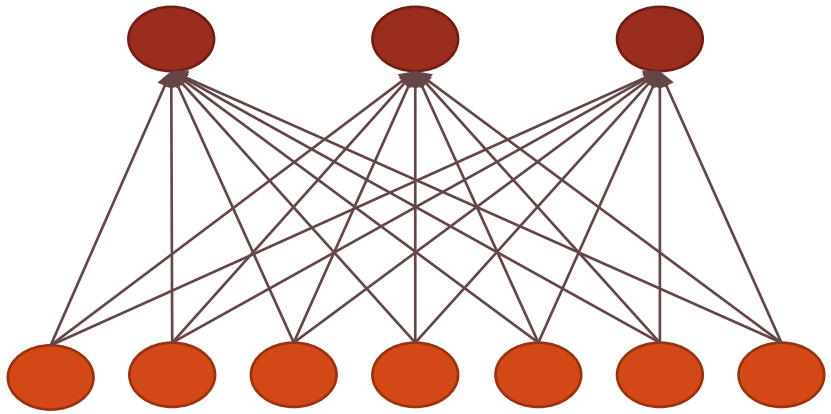


Global connectivity

Local connectivity

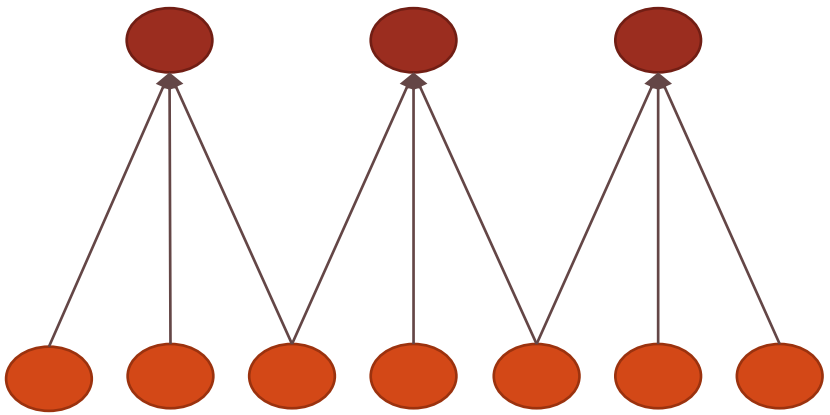
- # input units (neurons): 7
- # hidden units: 3
- **Number of parameters**
  - Global connectivity: ?
  - Local connectivity: ?

CNN: LOCAL CONNECTIVITY



Hidden layer

Input layer

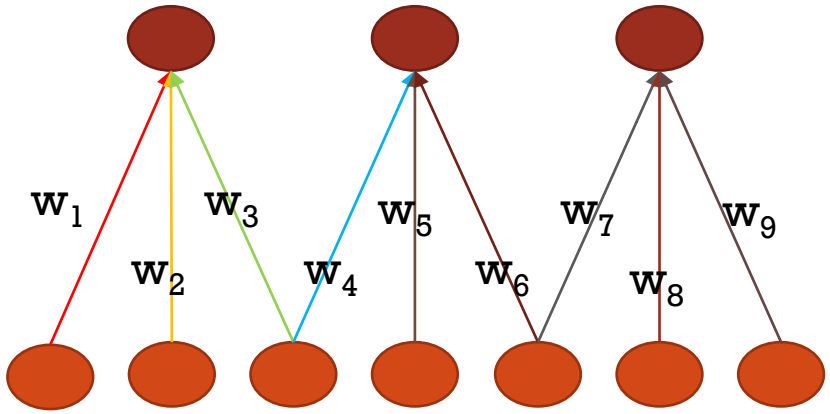


Global connectivity

Local connectivity

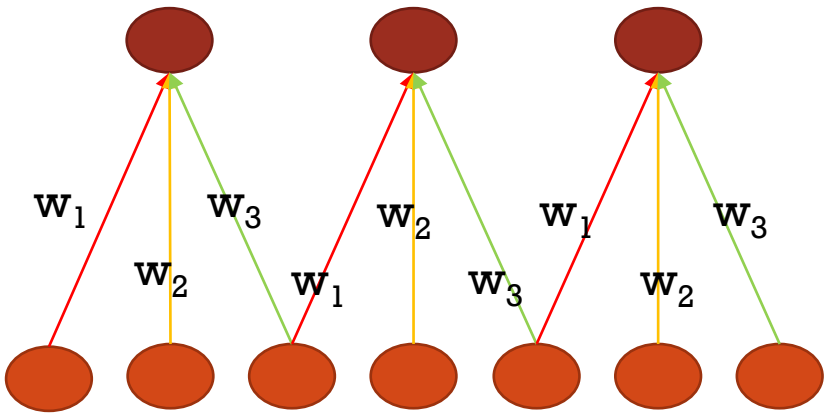
- # input units (neurons): 7
- # hidden units: 3
- **Number of parameters**
  - Global connectivity: **3 x 7 = 21**
  - Local connectivity: **3 x 3 = 9**

CNN: WEIGHT SHARING



Hidden layer

Input layer

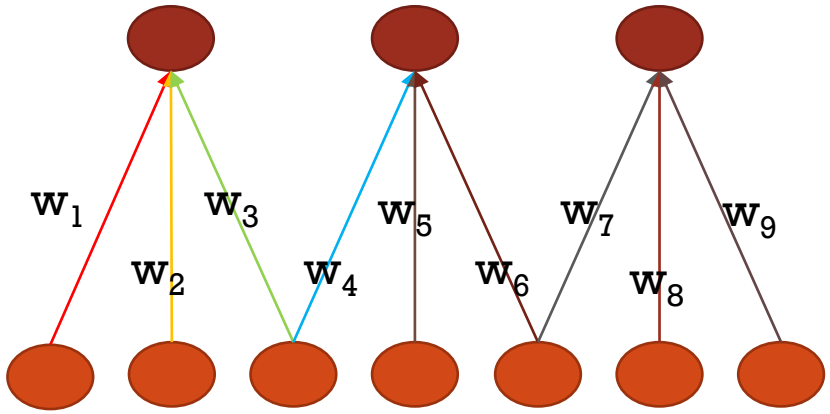


Without weight sharing

With weight sharing

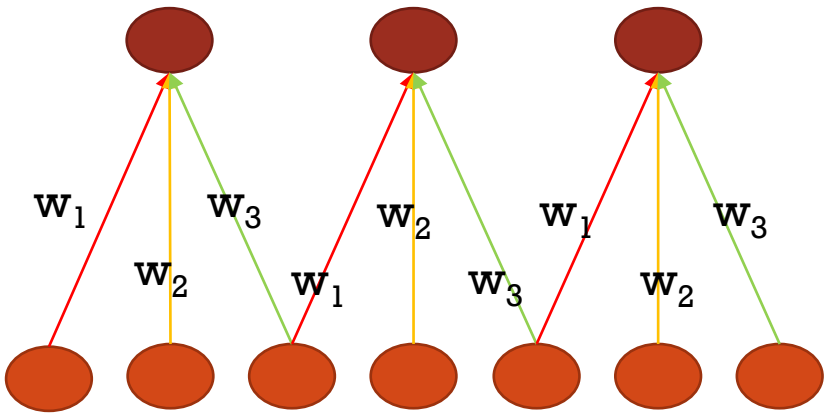
- # input units (neurons): 7
- # hidden units: 3

CNN: WEIGHT SHARING



Hidden layer

Input layer

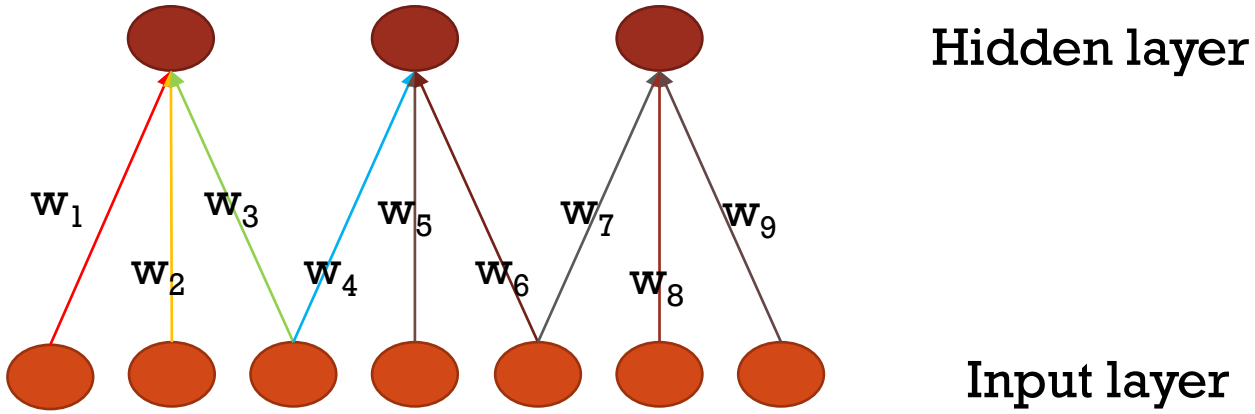


Without weight sharing

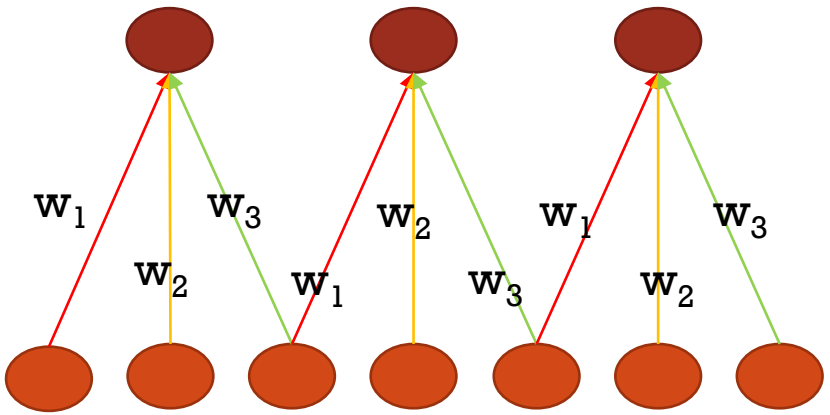
With weight sharing

- # input units (neurons): 7
- # hidden units: 3
- **Number of parameters**
  - Without weight sharing: ?
  - With weight sharing : ?

CNN: WEIGHT SHARING



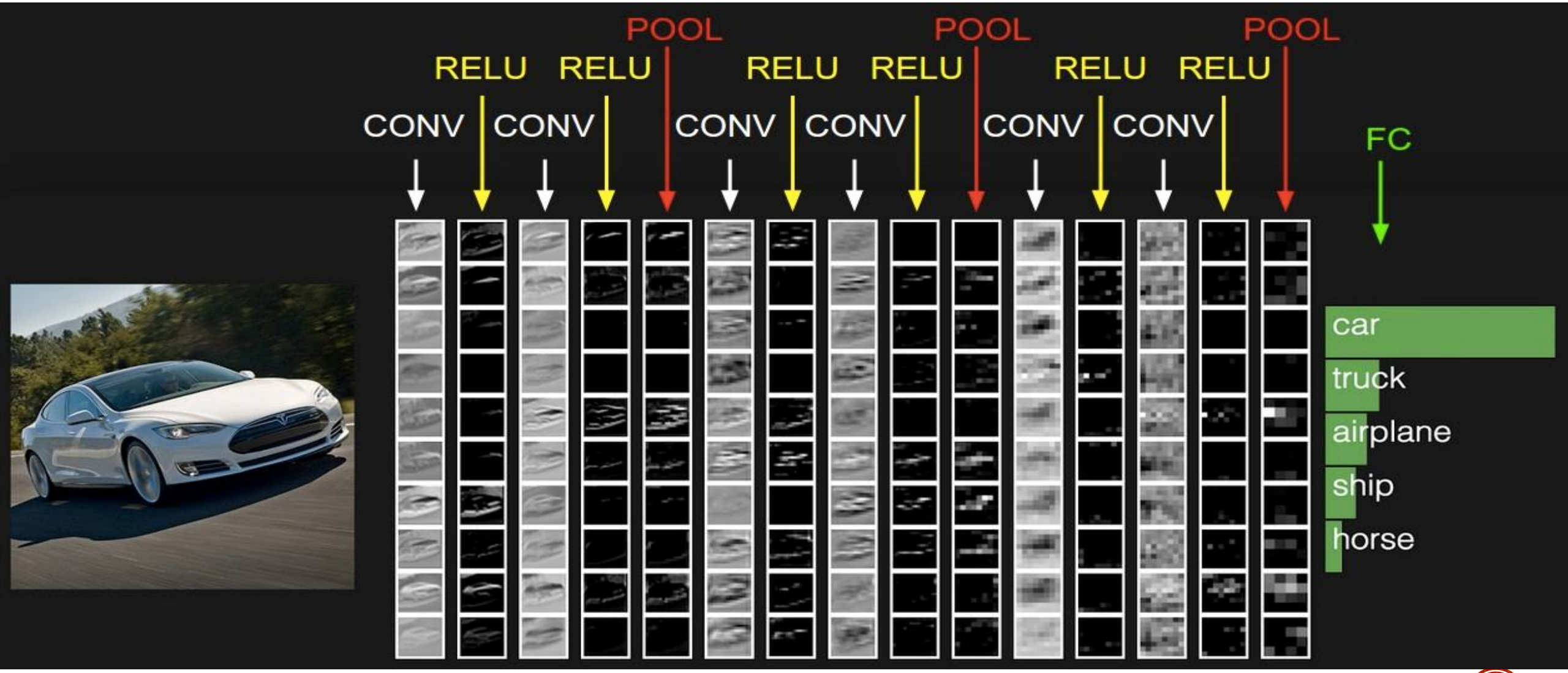
Without weight sharing



With weight sharing

- # input units (neurons): 7
- # hidden units: 3
- **Number of parameters**
  - Without weight sharing: **3 x 3 = 9**
  - With weight sharing : **3 x 1 = 3**

CONVOLUTIONAL NEURAL NETWORKS



Source: cs231n, Stanford University

LAYERS USED TO BUILD CONVNETS

Input Layer (Input image)

Convolutional Layer

Non-linearity Layer (such as Sigmoid, Tanh, ReLU, PReLU, ELU, Swish, etc.)

Pooling Layer (such as Max Pooling, Average Pooling, etc.)

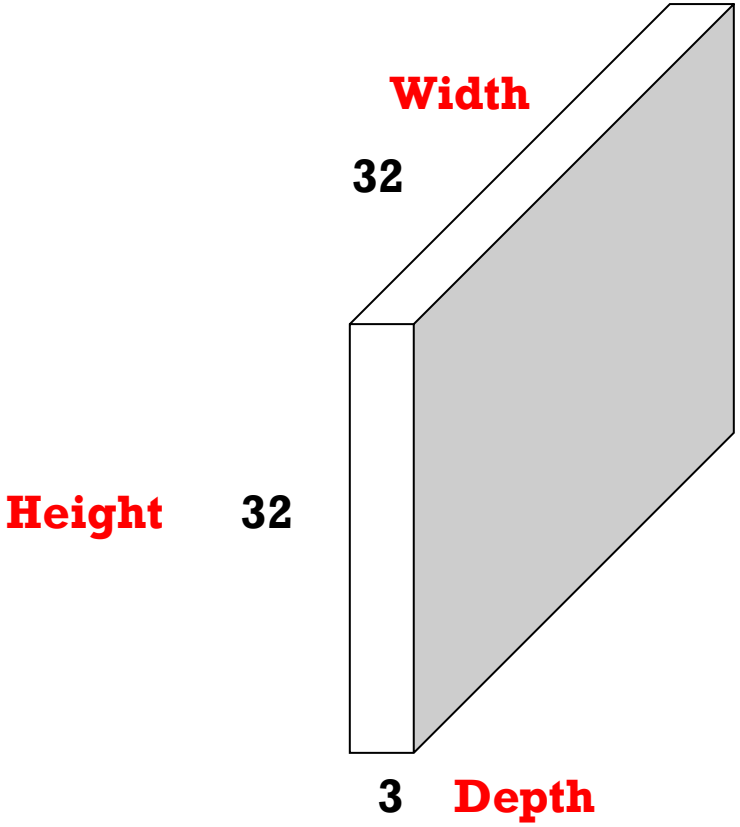
Fully-Connected Layer

Classification Layer (Softmax, etc.)

CONVOLUTIONAL LAYER

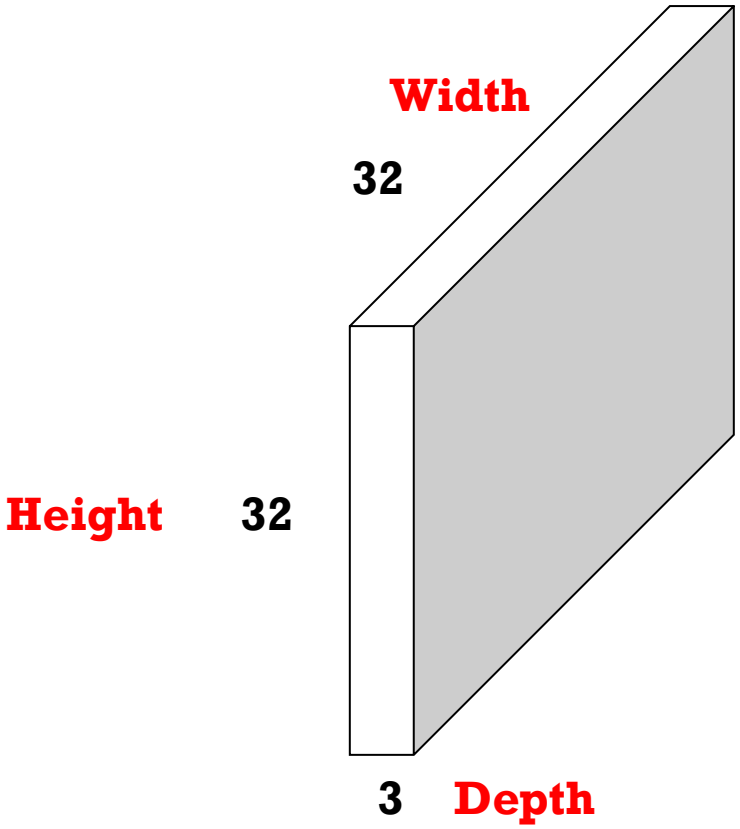
32×32×3 Image

-> preserve spatial structure

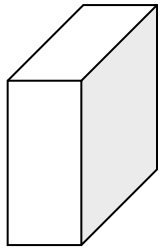


CONVOLUTIONAL LAYER

32×32×3 Image



5×5×3 Filter



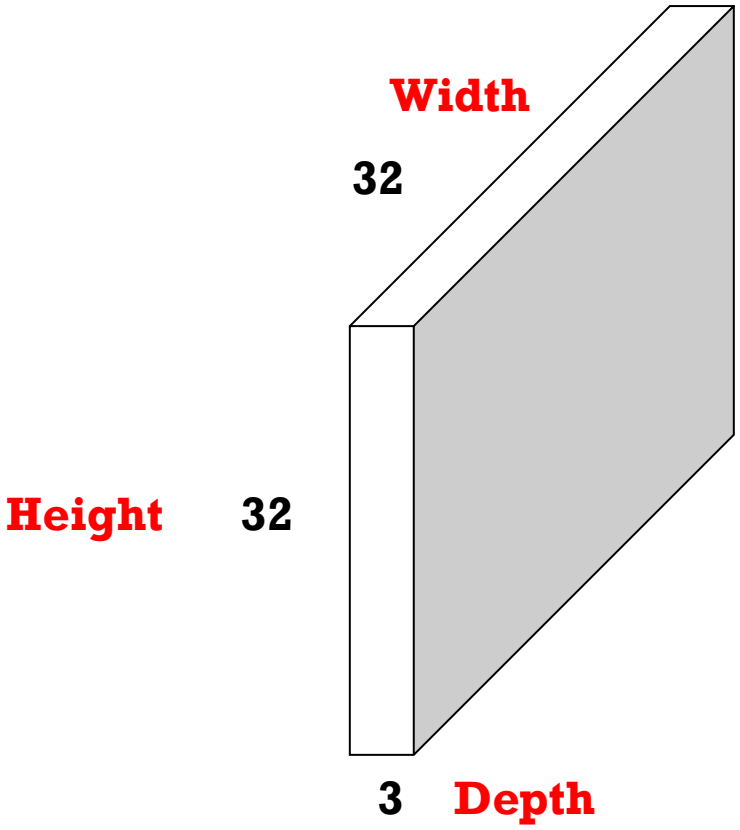
Convolve the filter with the image i.e. "slide over the image spatially, computing dot products"

CONVOLUTIONAL LAYER

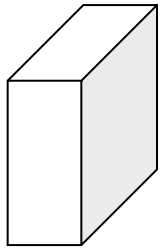
Handling multiple input channels

Filters always extend the full depth of the input volume

32×32×3 Image

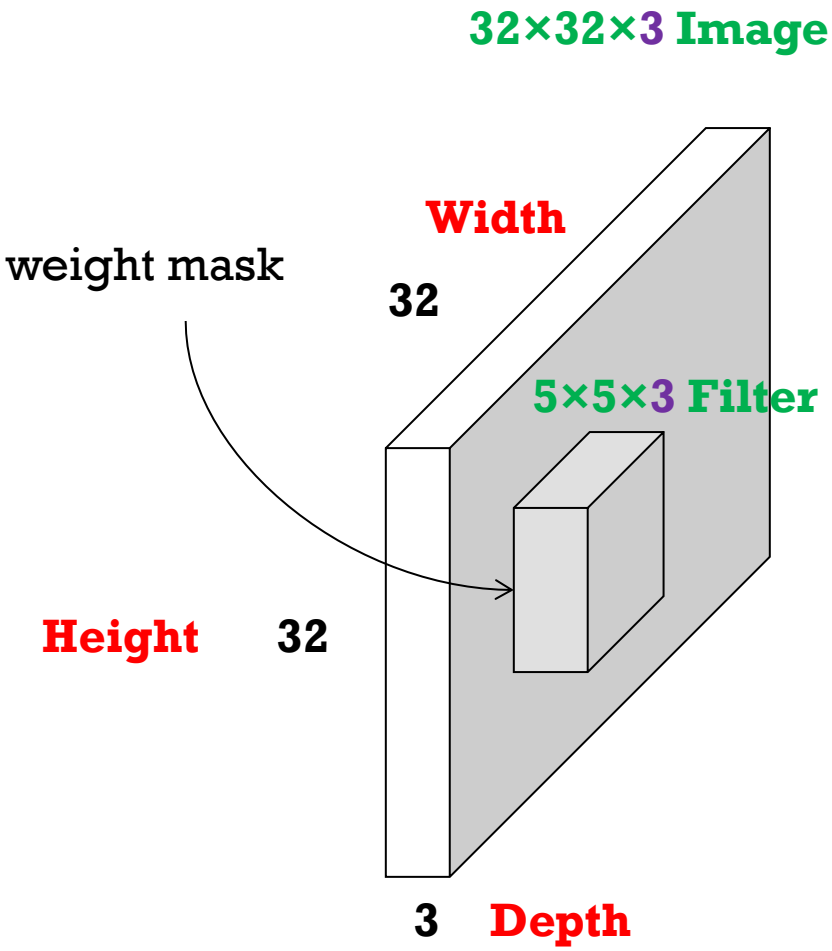


5×5×3 Filter

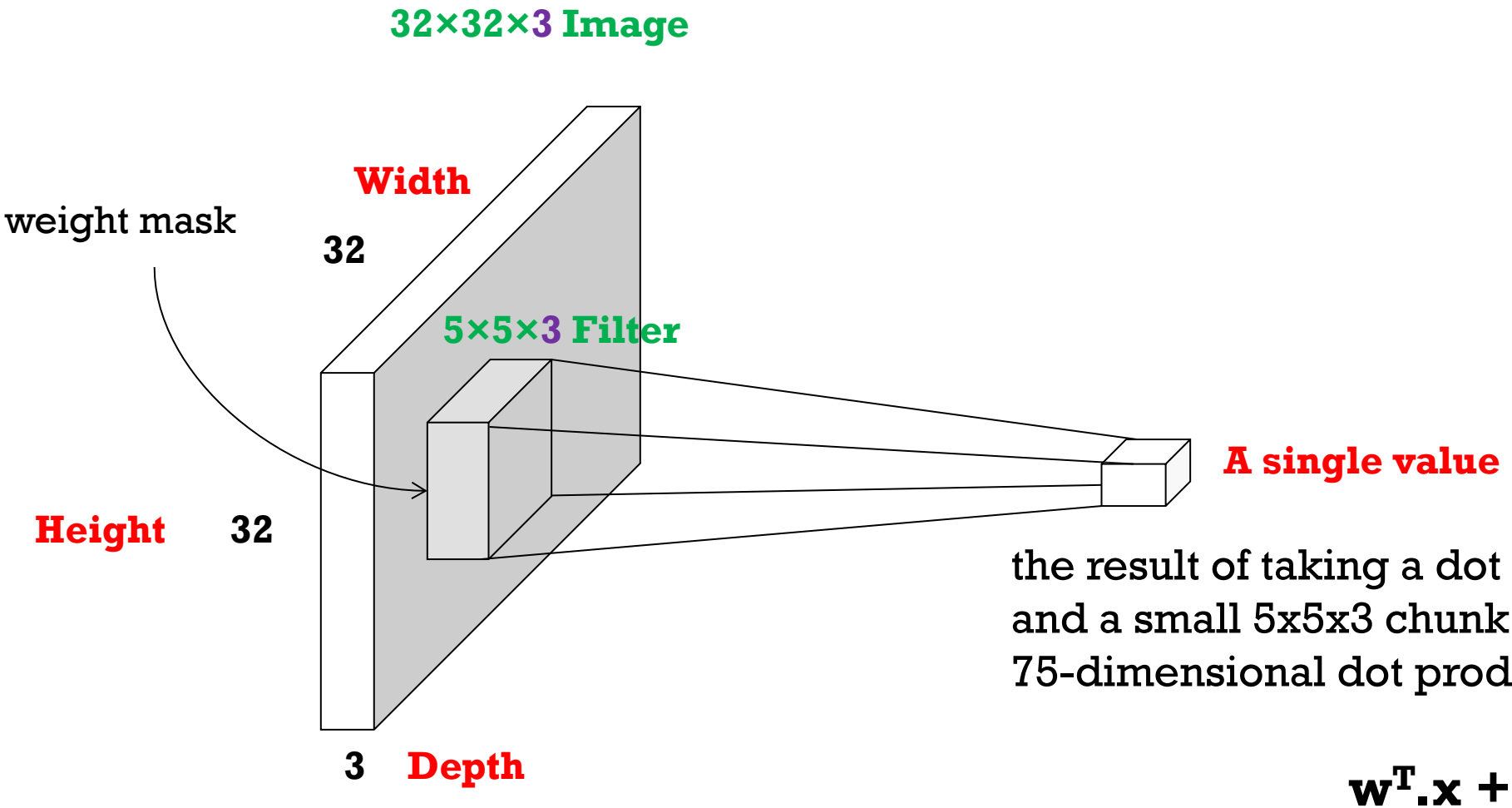


Convolve the filter with the image i.e. "slide over the image spatially, computing dot products"

CONVOLUTIONAL LAYER



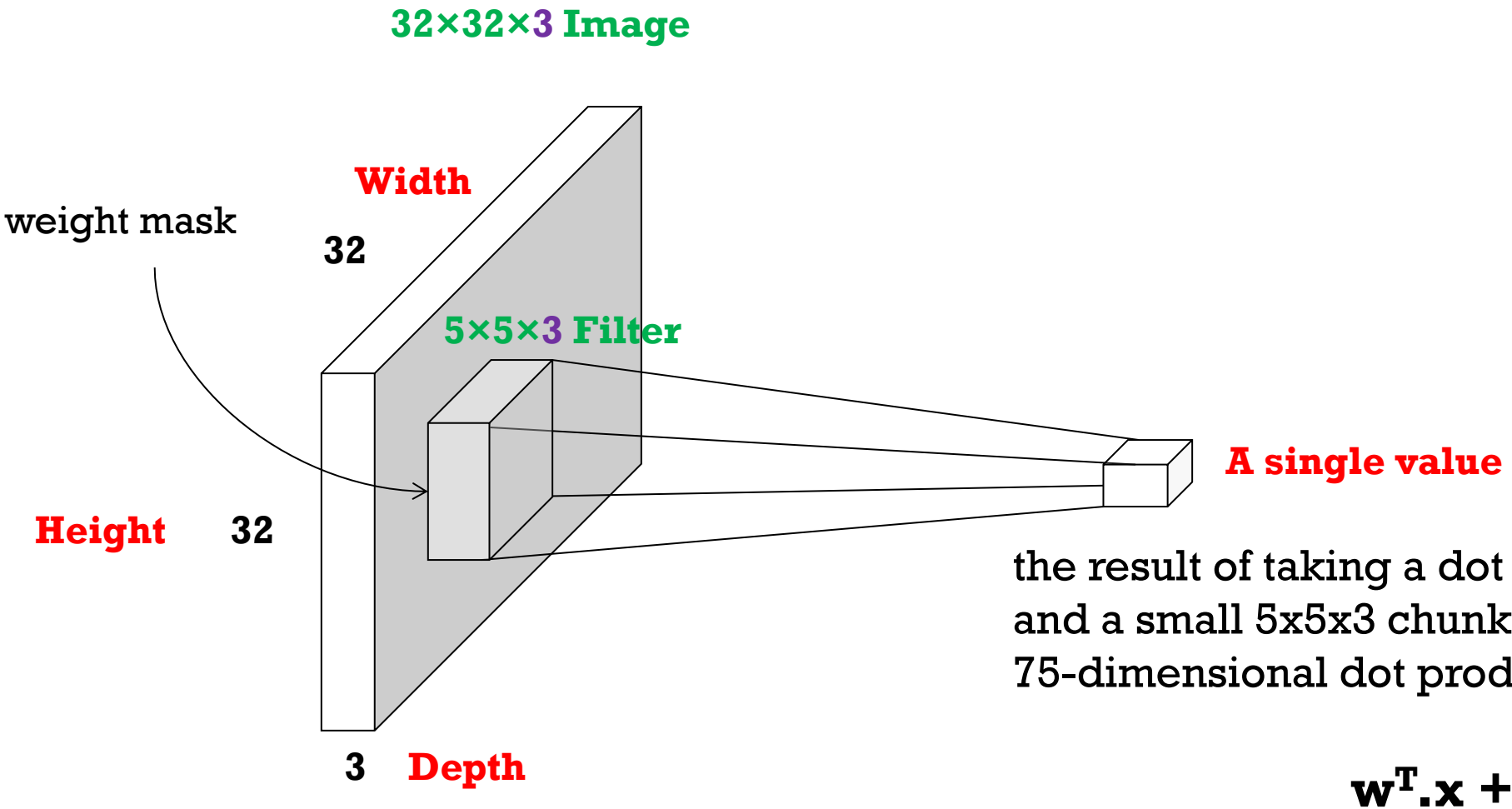
CONVOLUTIONAL LAYER

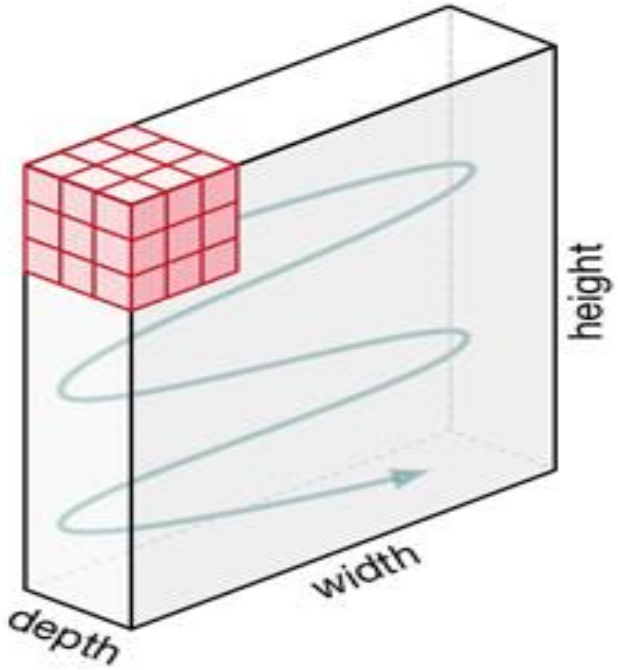


the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T.x + b$$

CONVOLUTIONAL LAYER

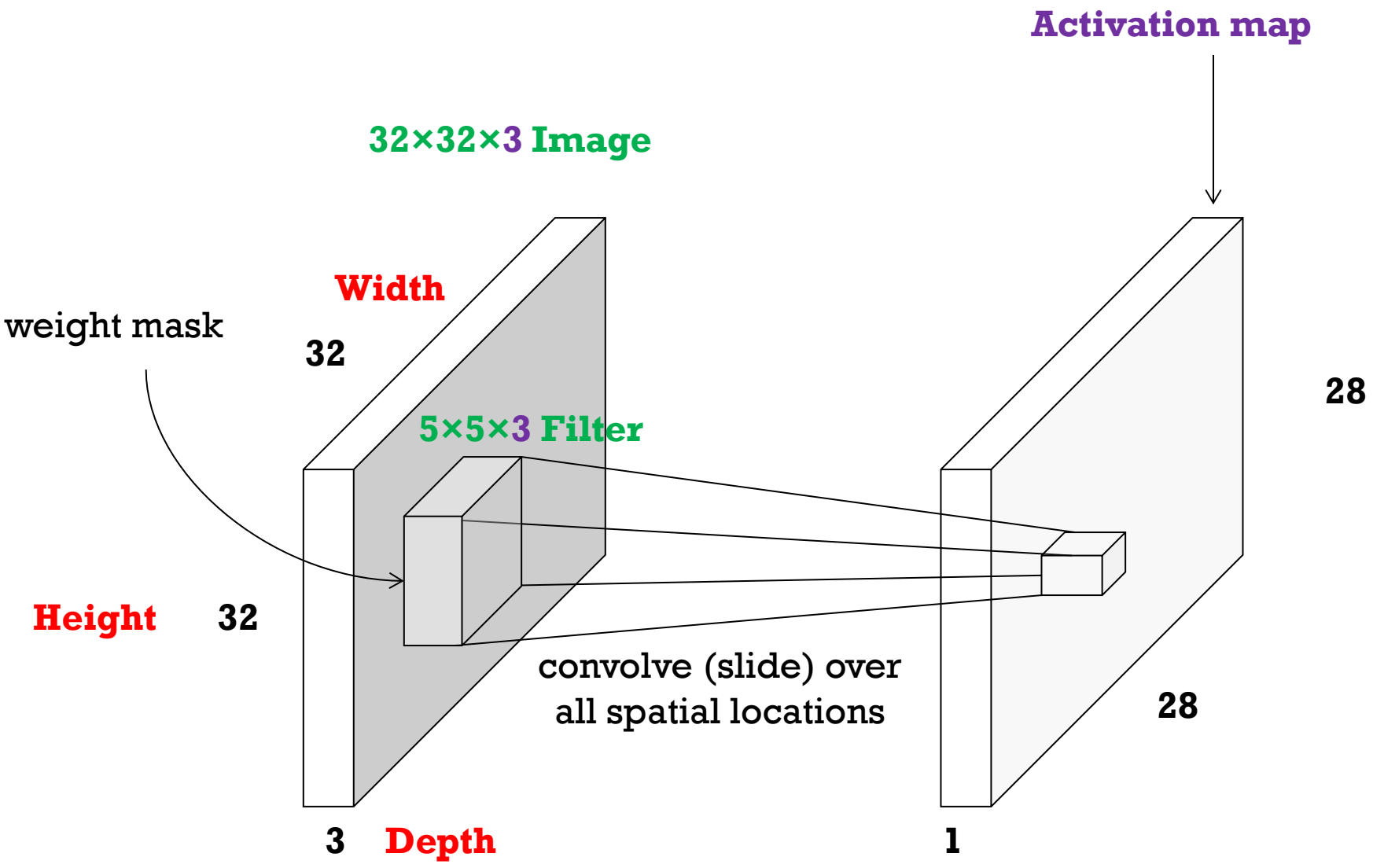


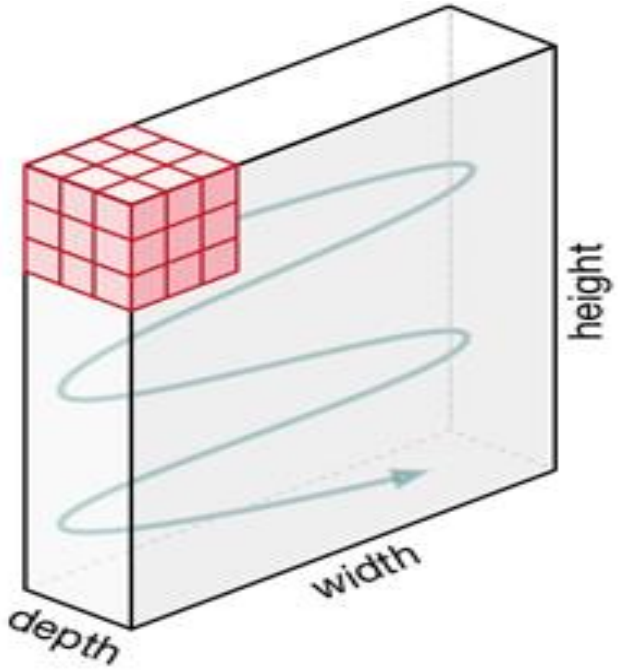


the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T.x + b$$

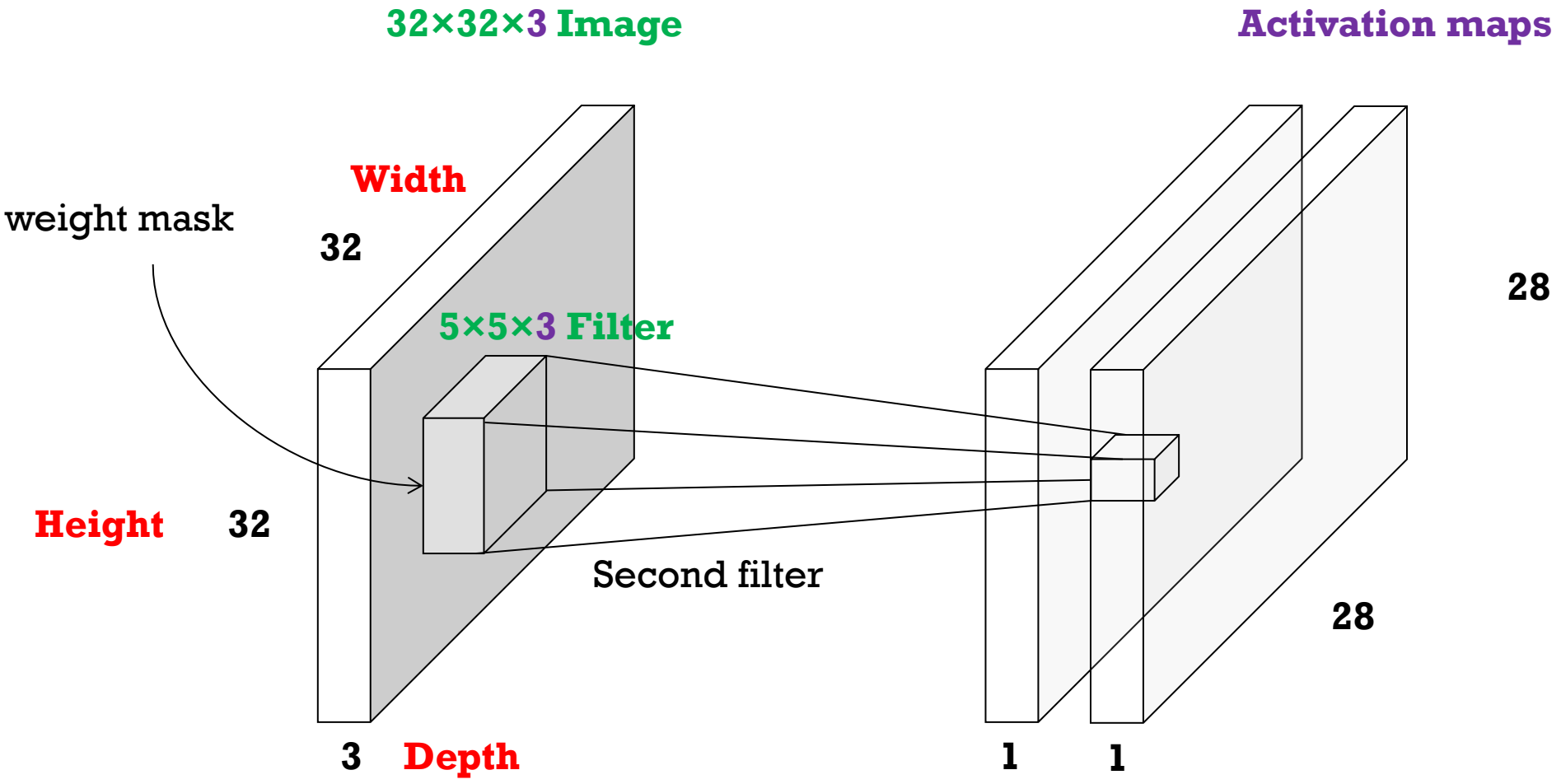
CONVOLUTIONAL LAYER





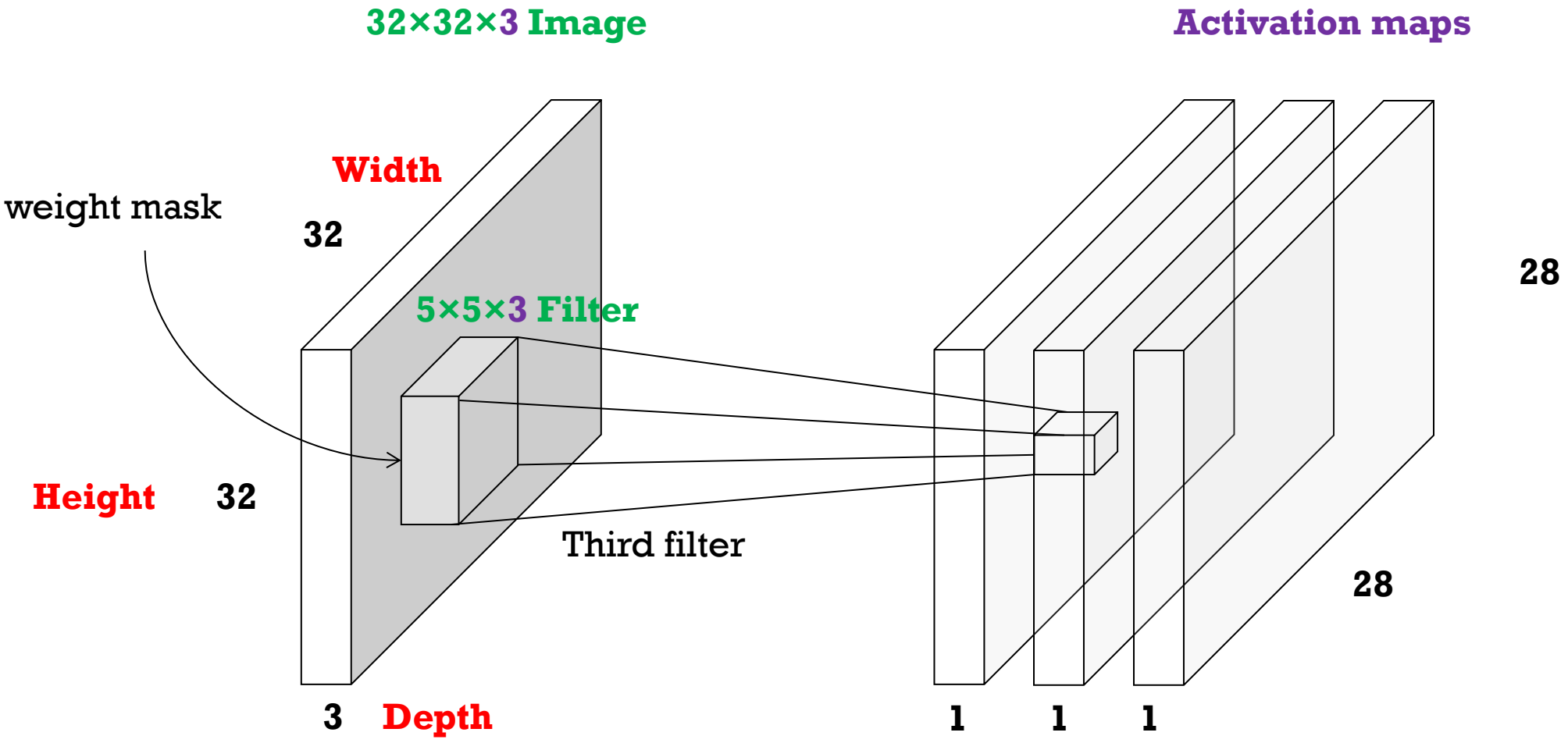
CONVOLUTIONAL LAYER

Handling multiple output maps



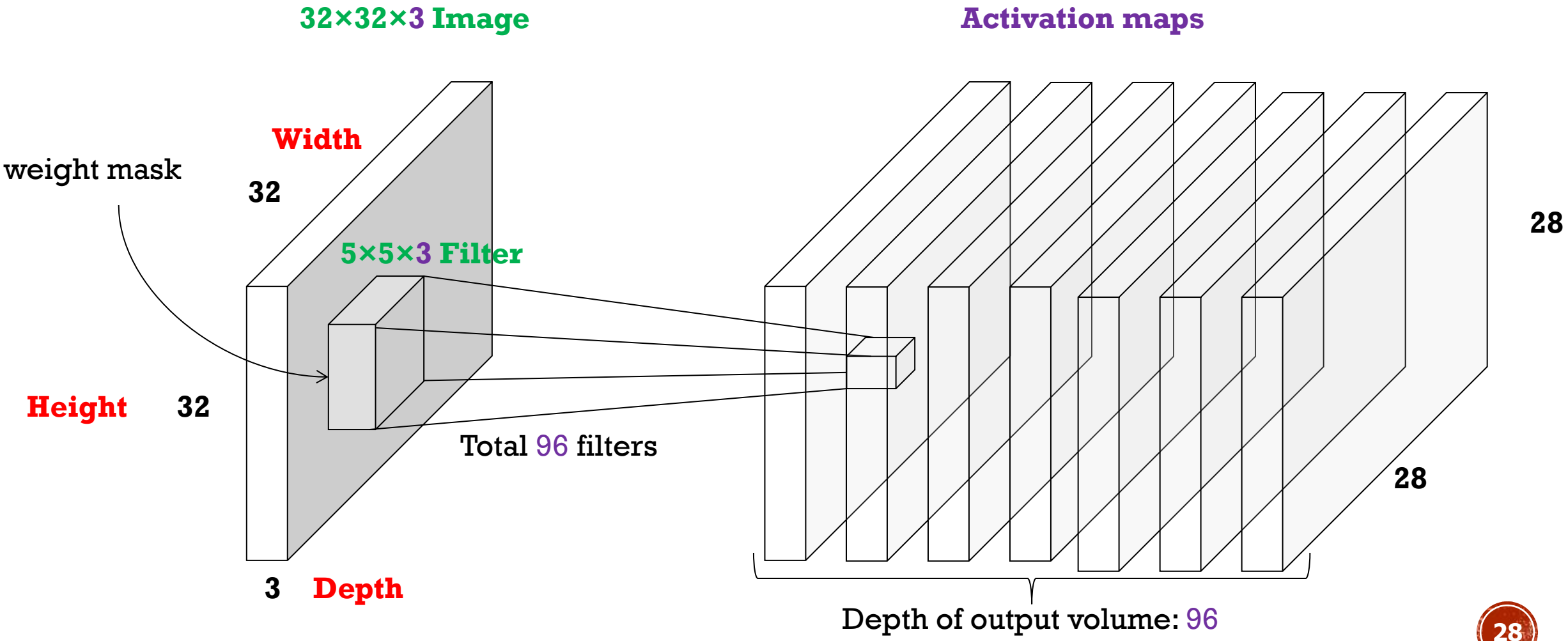
CONVOLUTIONAL LAYER

Handling multiple output maps



CONVOLUTIONAL LAYER

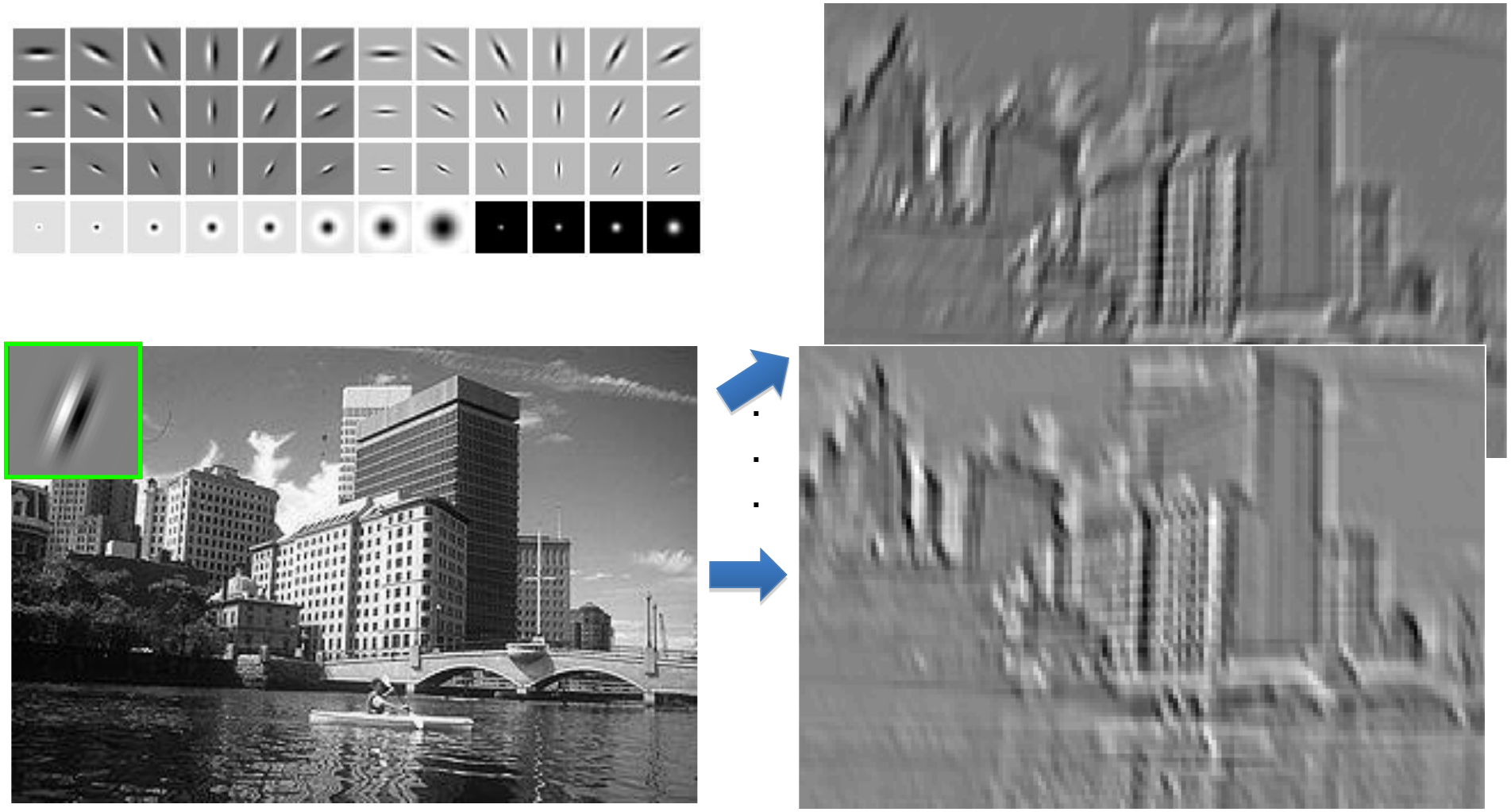
Handling multiple output maps



CONVOLUTION AND TRADITIONAL FEATURE EXTRACTION

bank of K filters

K feature maps



image

feature map

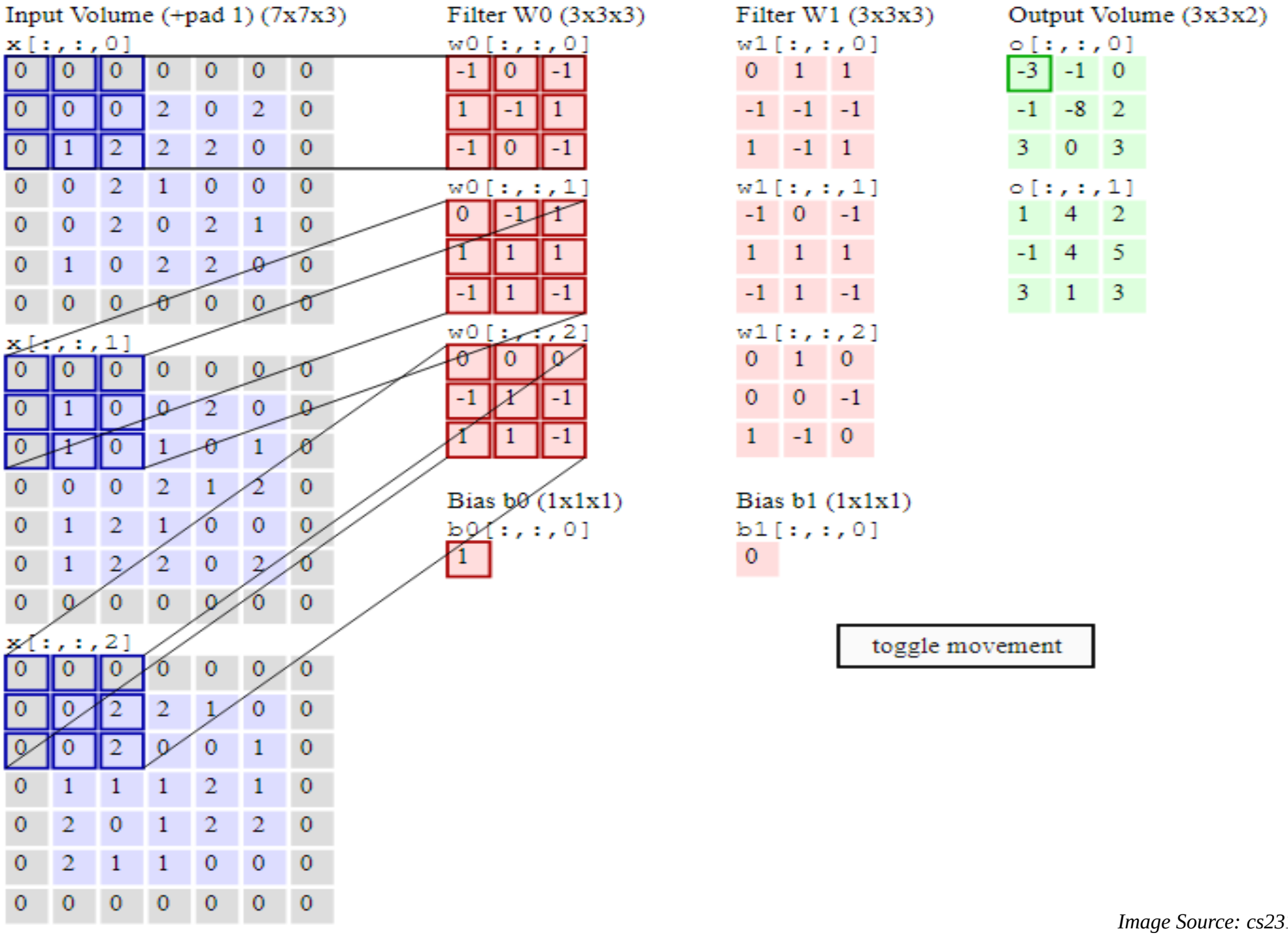


Image Source: cs231n, Oxford University

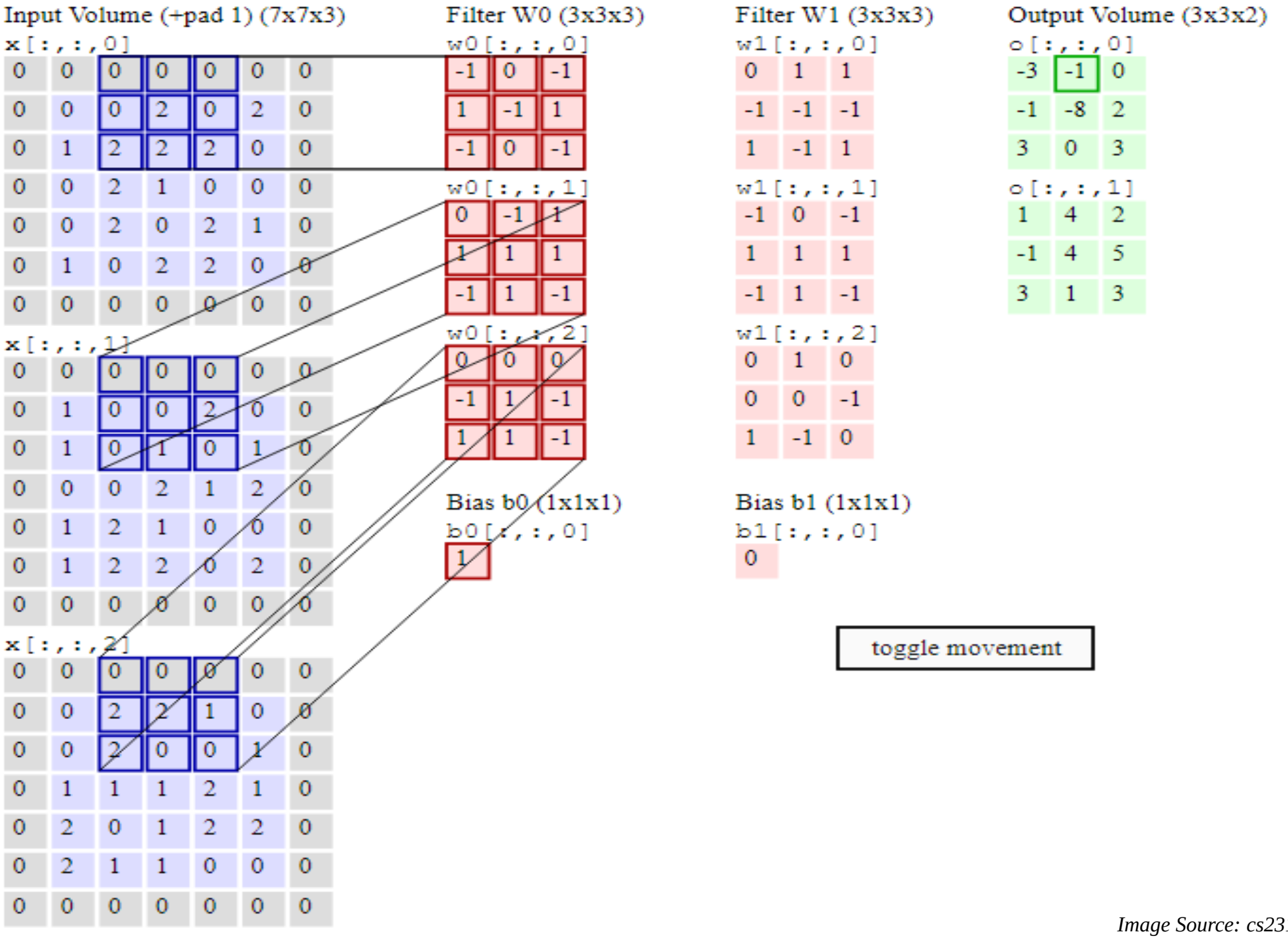


Image Source: cs231n, Oxford University

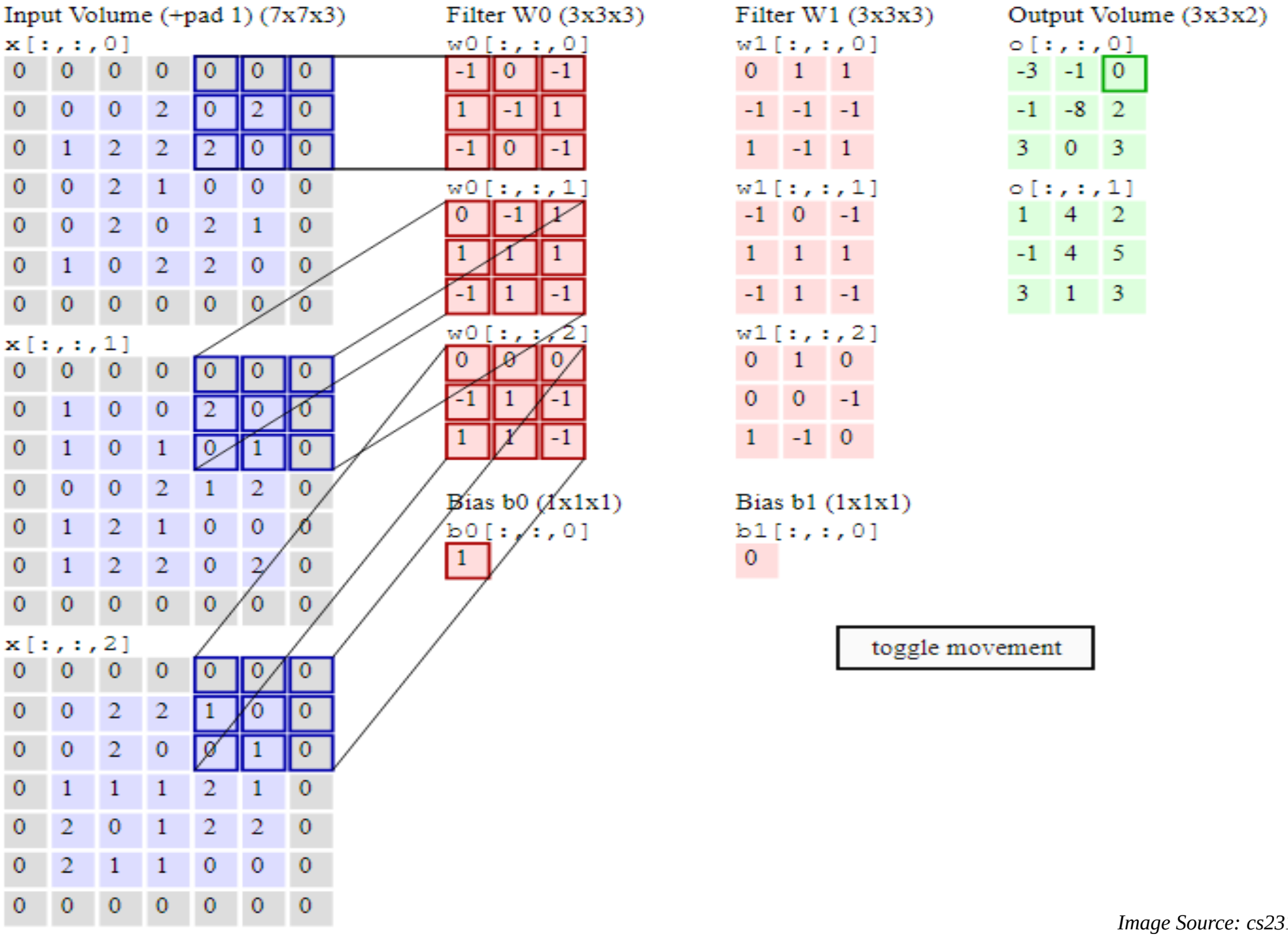
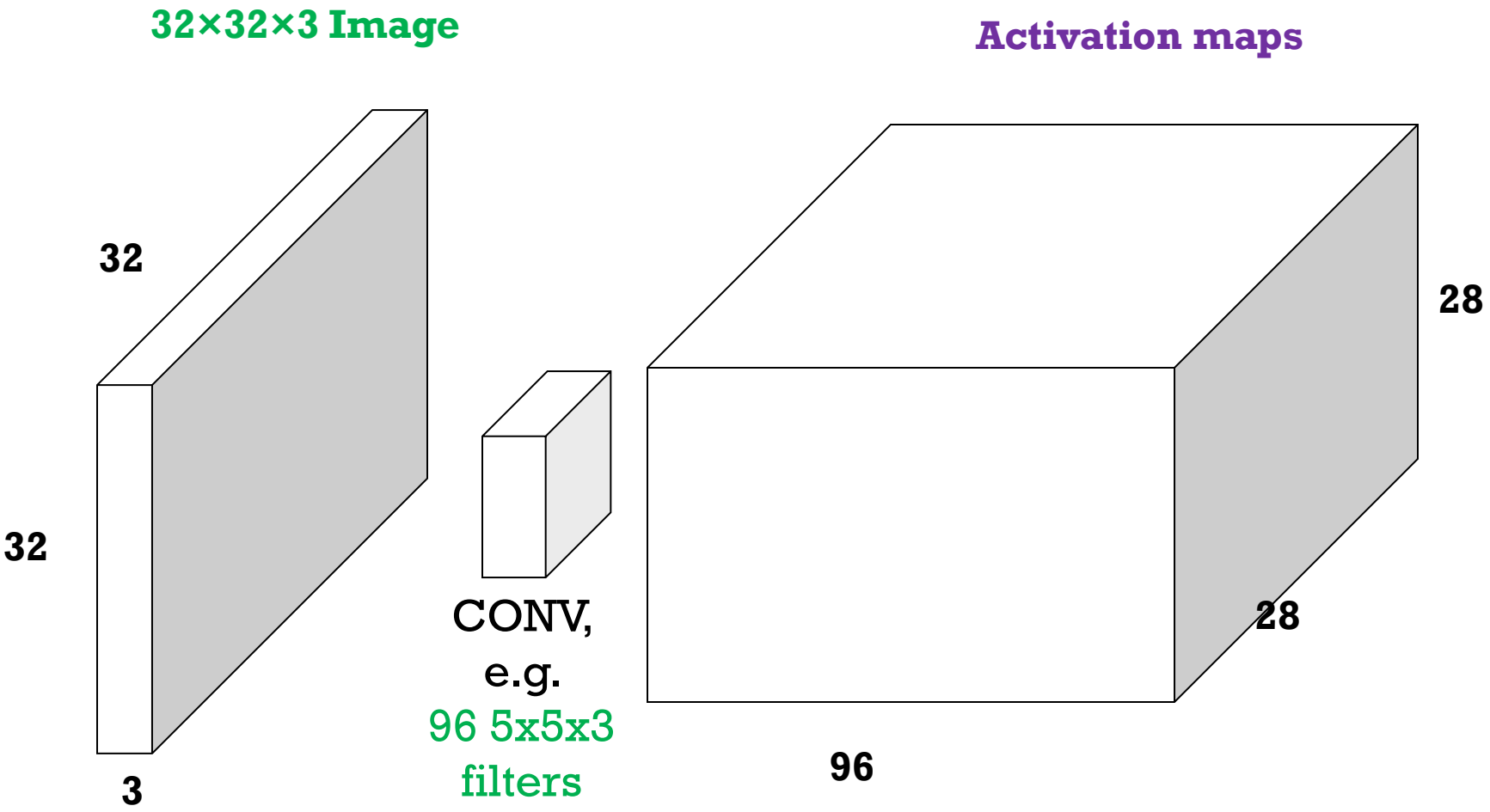


Image Source: cs231n, Oxford University

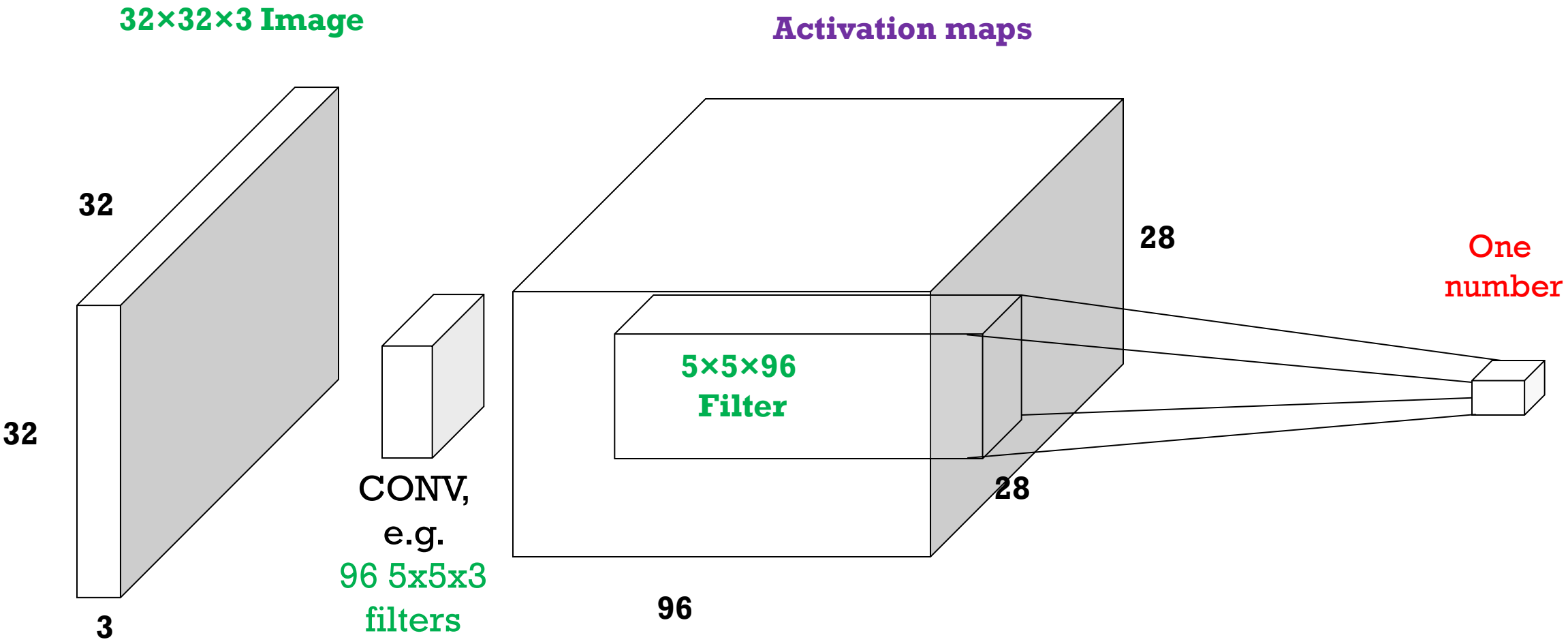
CONVOLUTIONAL LAYER

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



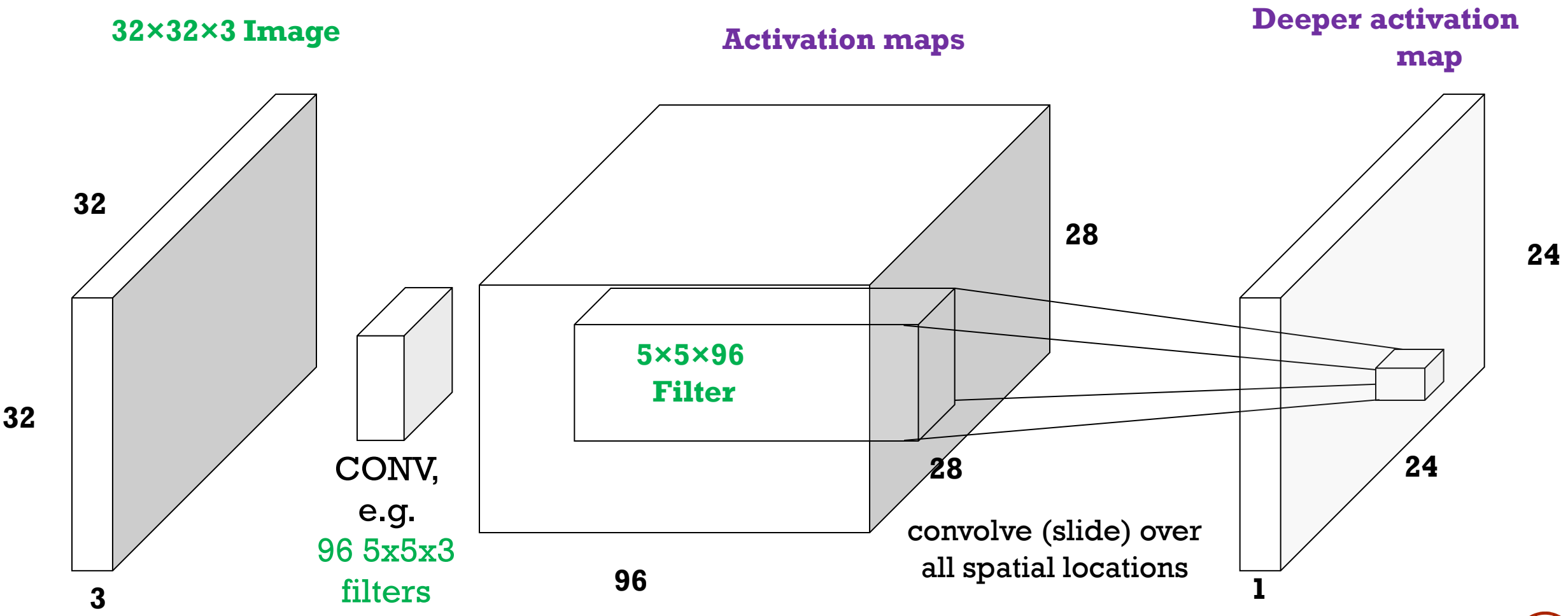
CONVOLUTIONAL LAYER

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



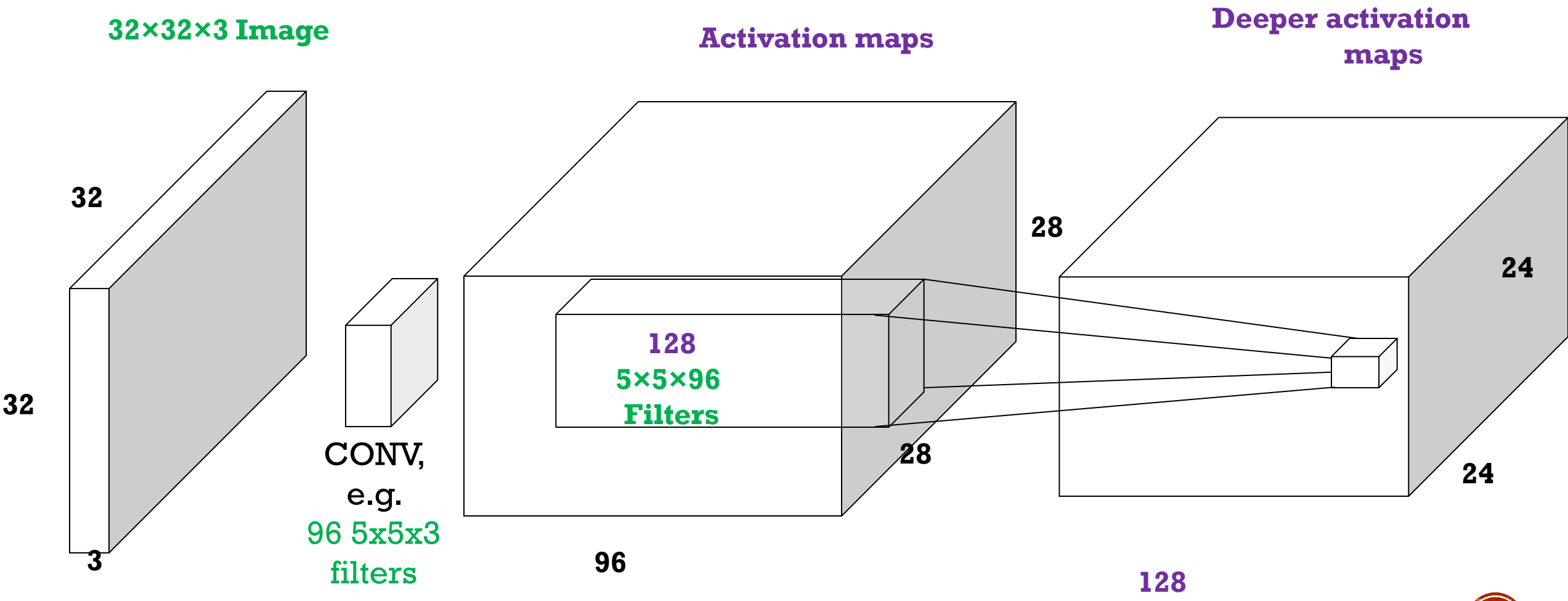
CONVOLUTIONAL LAYER

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



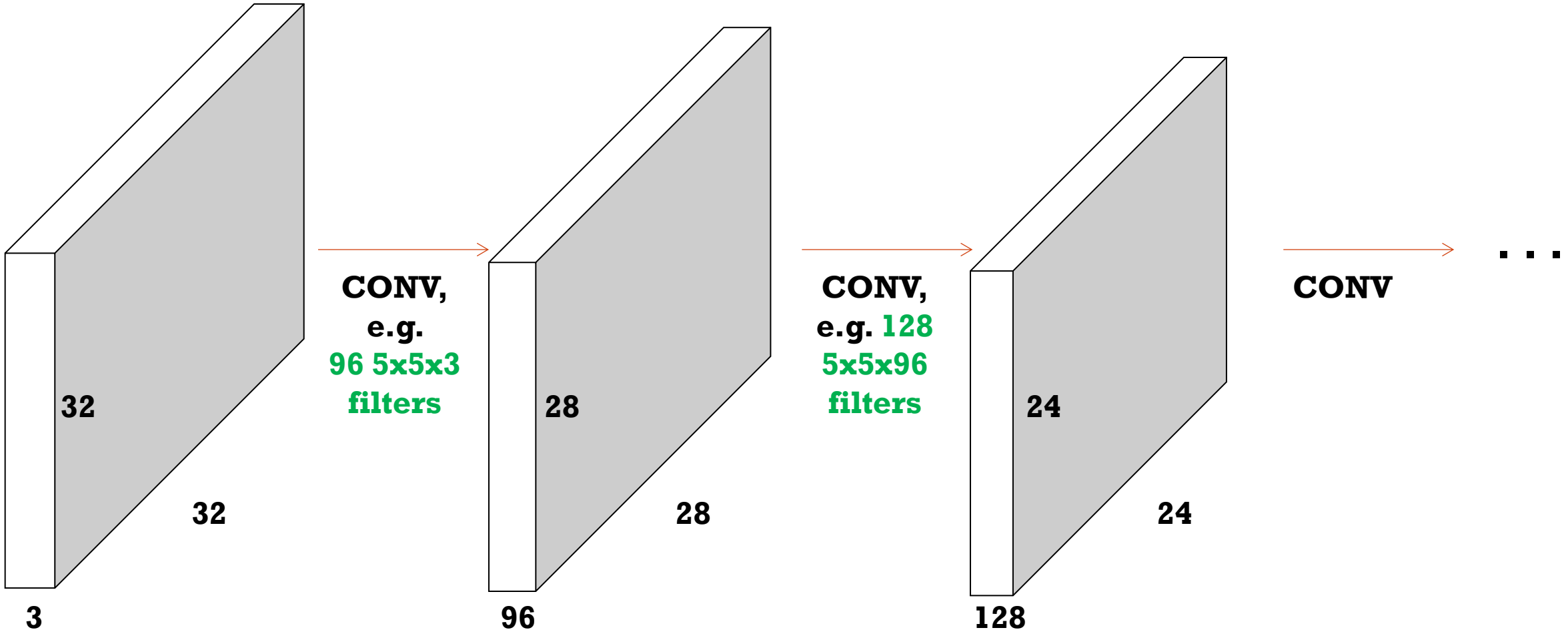
CONVOLUTIONAL LAYER

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



MULTILAYER CONVOLUTION

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



ANY CONVOLUTION LAYER

- Local connectivity
- Weight sharing
- Handling multiple input channels
- Handling multiple output maps

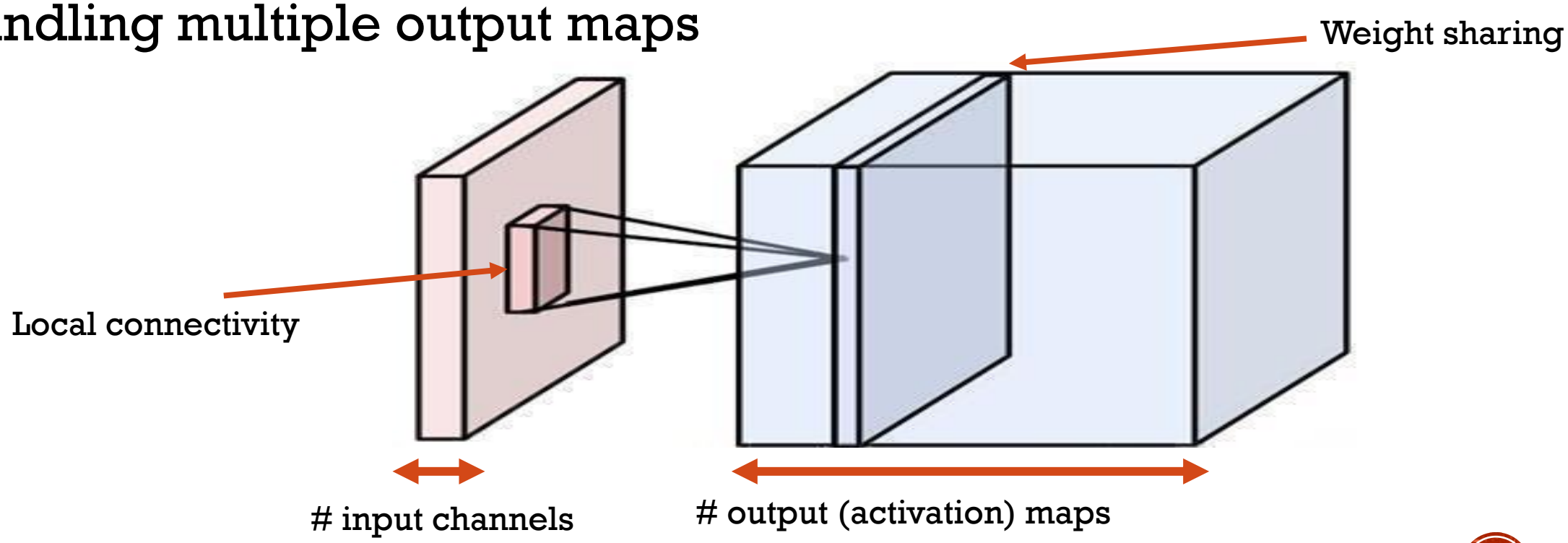
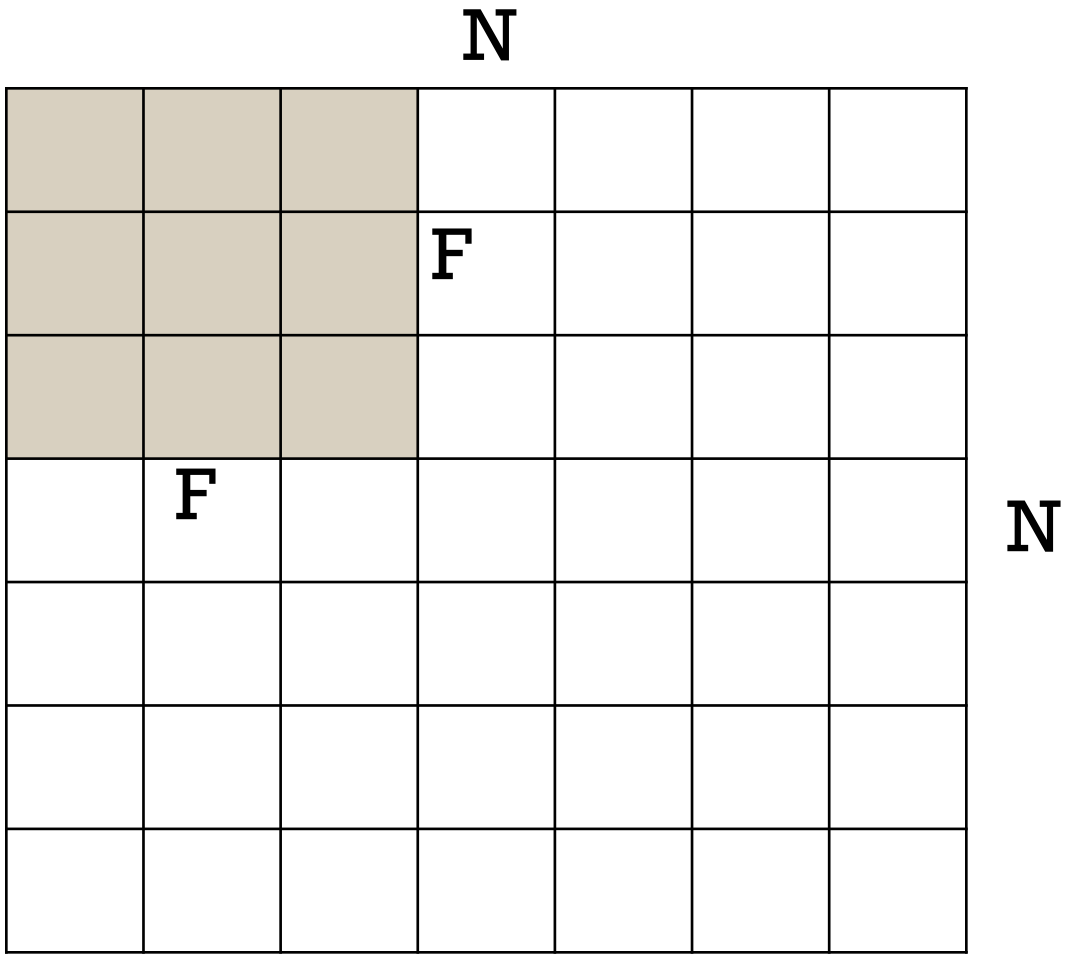


Image credit: A. Karpathy

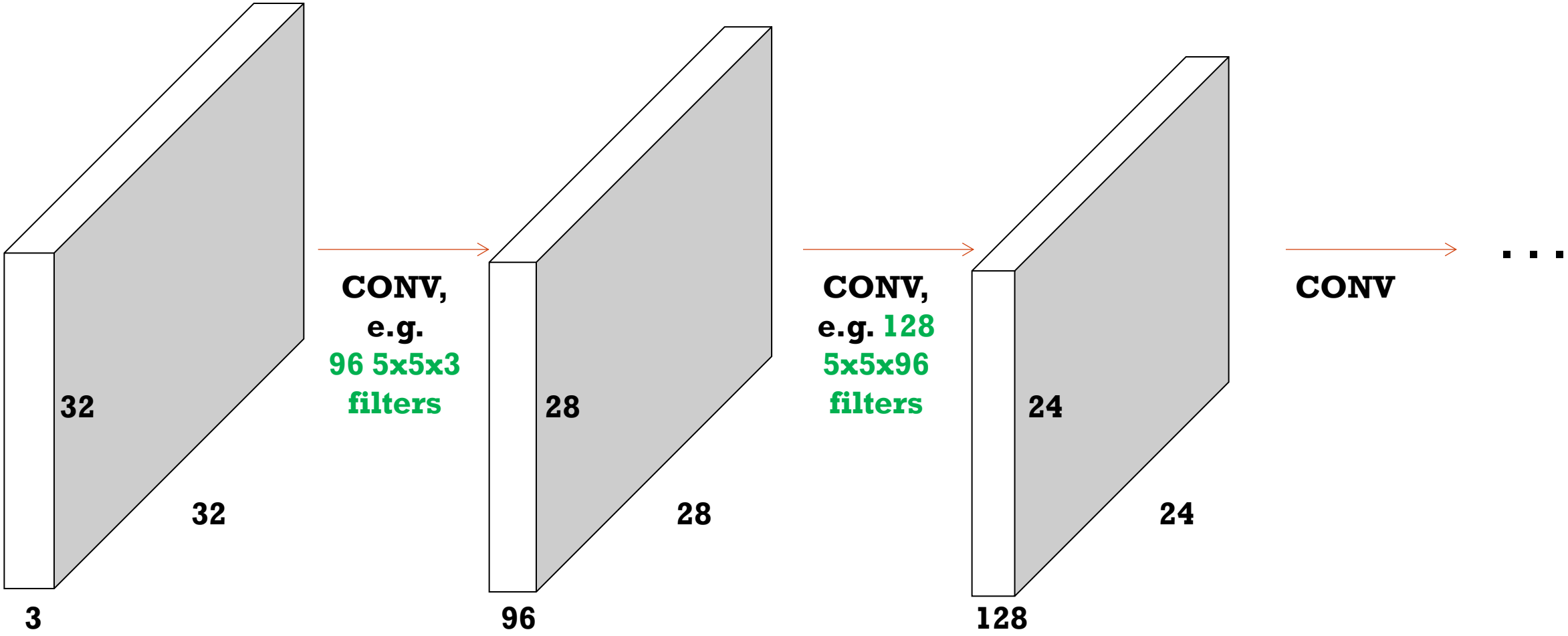
A CLOSER LOOK AT SPATIAL DIMENSIONS



Output size
(N - F) / stride + 1

e.g. N = 7, F = 3
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33

A CLOSER LOOK AT SPATIAL DIMENSIONS



E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.

Source: cs231n, Stanford University

IN PRACTICE: COMMON TO ZERO PAD

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

e.g. input 7×7 (spatially)
3×3 filter, applied with stride 1
pad with 1 pixel border

7×7 Output

in general, common to see CONV layers with stride 1, filters of size F×F, and zero-padding with (F-1)/2. (will preserve size spatially)

e.g.
F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

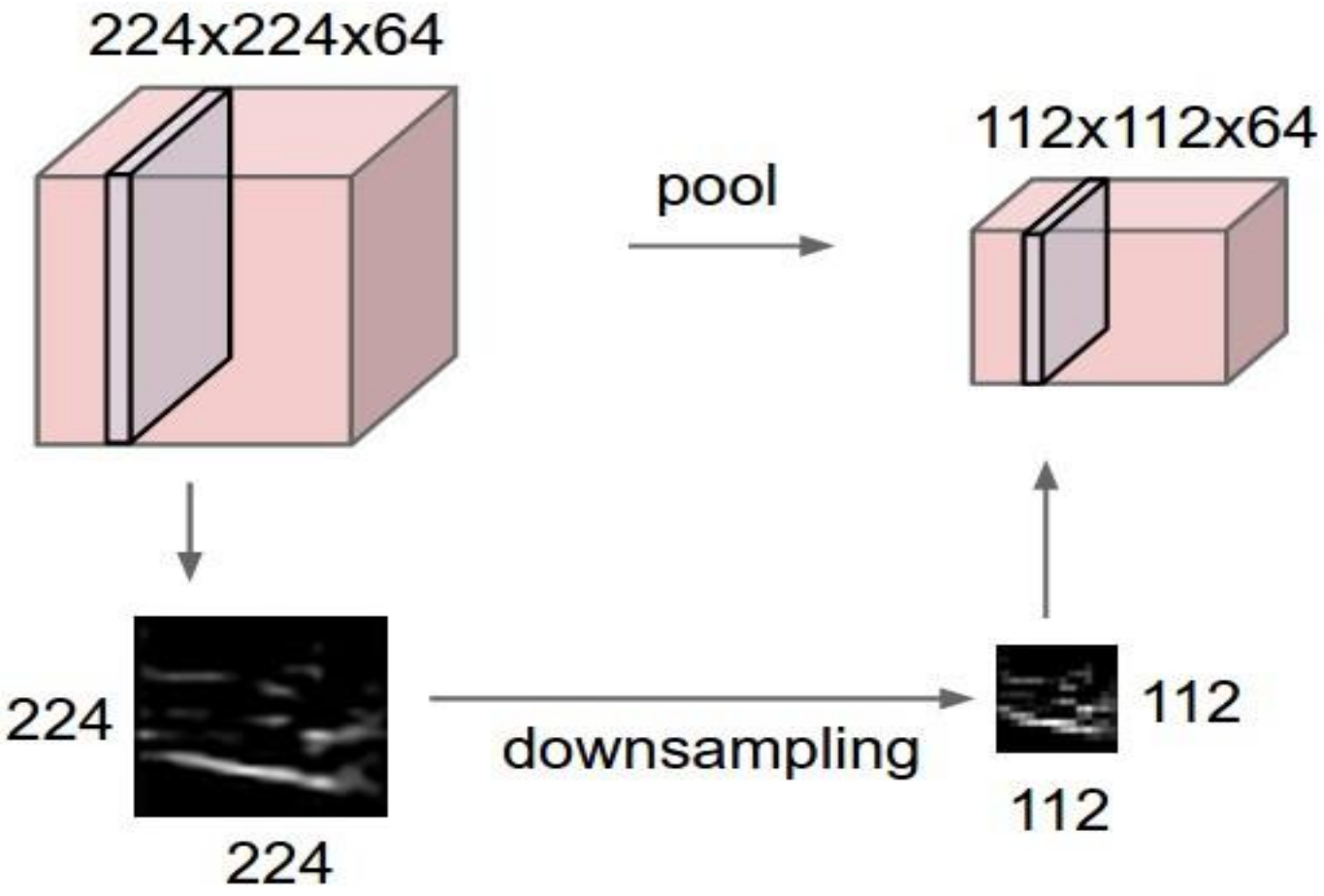
Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.

Source: cs231n, Stanford University

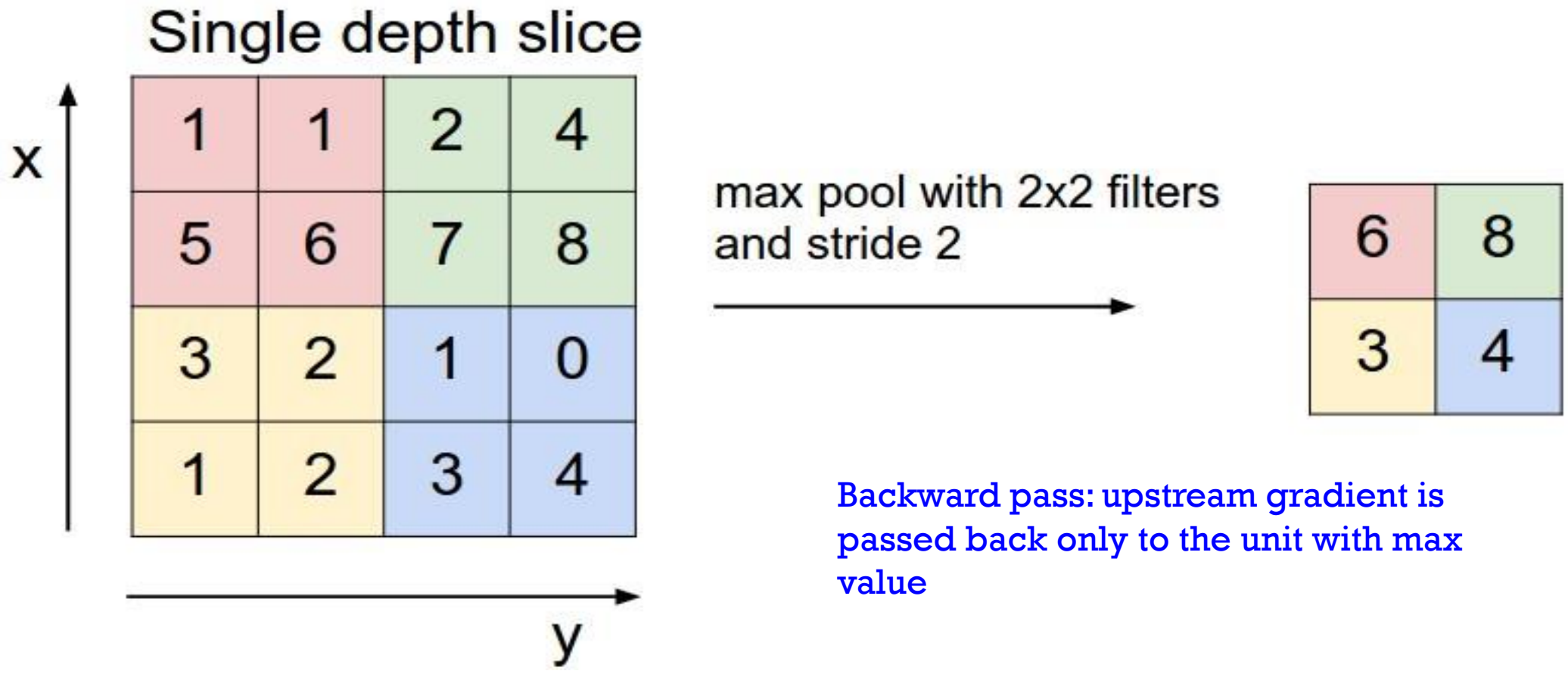
POOLING LAYER

- makes the representations smaller and more manageable
- operates over each activation map independently:



Source: cs231n, Stanford University

MAX POOLING



Backward pass: upstream gradient is passed back only to the unit with max value

Source: cs231n, Stanford University

POOLING LAYER

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires two hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

Source: cs231n, Stanford University

FULLY CONNECTED LAYER

- Connect every neuron in one layer to every neuron in another layer
- Same as the traditional multi-layer perceptron neural network

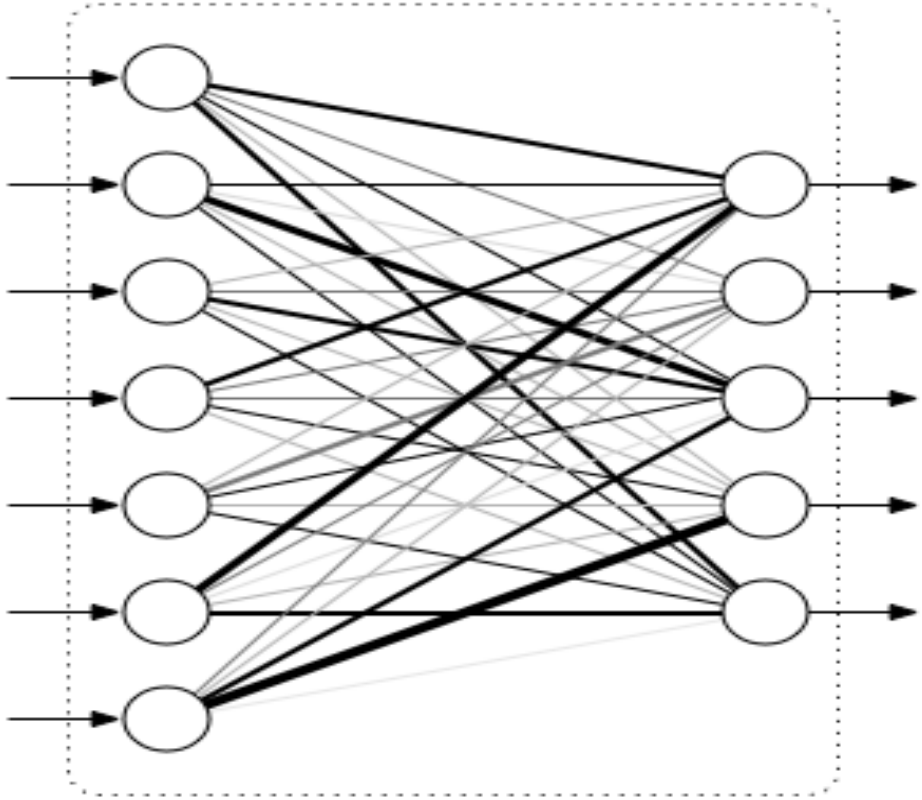


Image Source: machinethink.net

FULLY CONNECTED LAYER

- Connect every neuron in one layer to every neuron in another layer
- Same as the traditional multi-layer perceptron neural network

No. of Neurons (Last FC)
= No. of classes

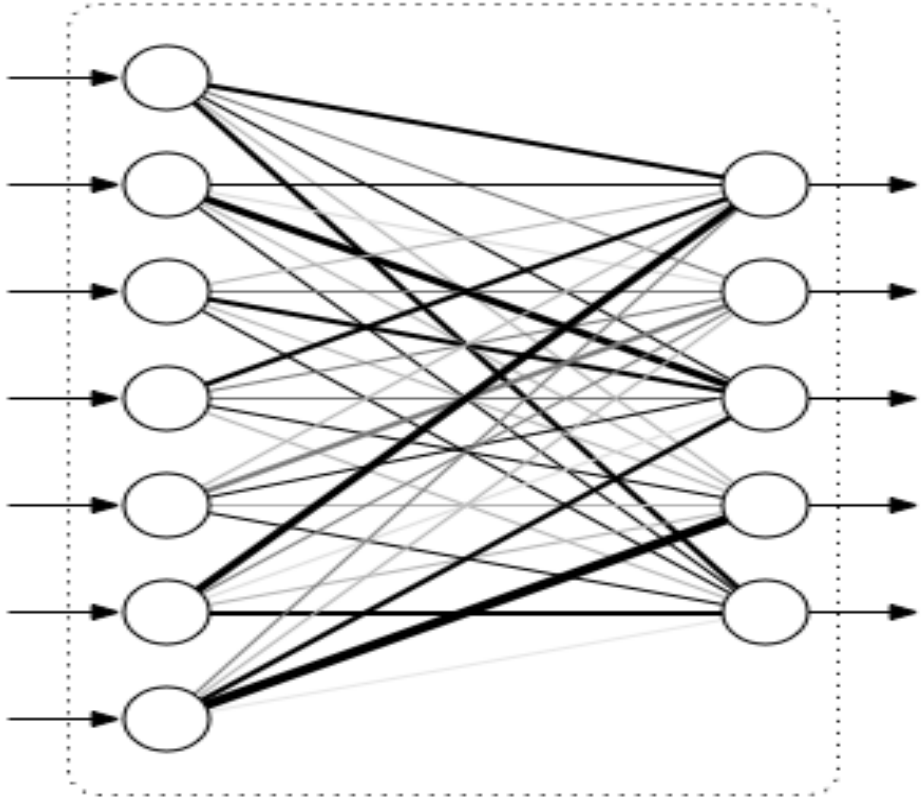


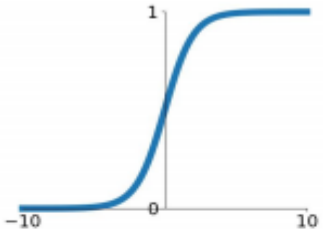
Image Source: machinethink.net

NON-LINEARITY LAYER

Activation Functions

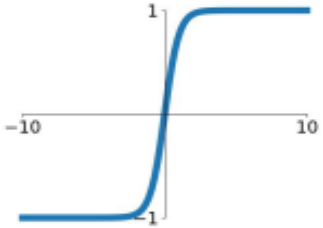
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



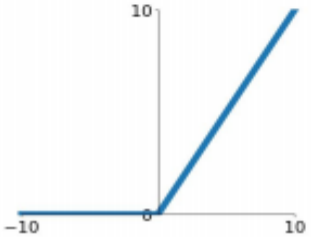
tanh

$$\tanh(x)$$



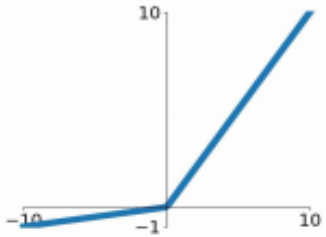
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

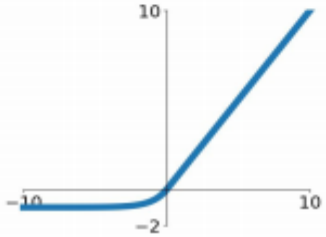


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

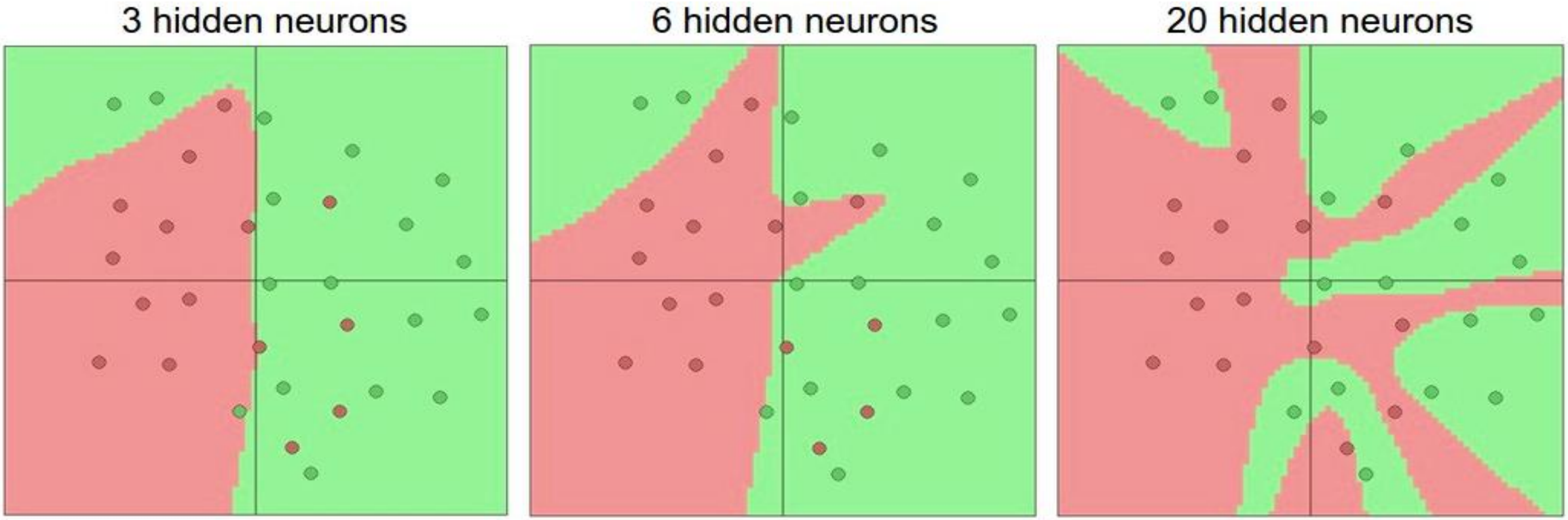
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Source: cs231n, Stanford University

Activation Functions

Non-linearities needed to learn complex (non-linear) representations of data, otherwise the NN would be just a linear function



More layers and neurons can approximate more complex functions

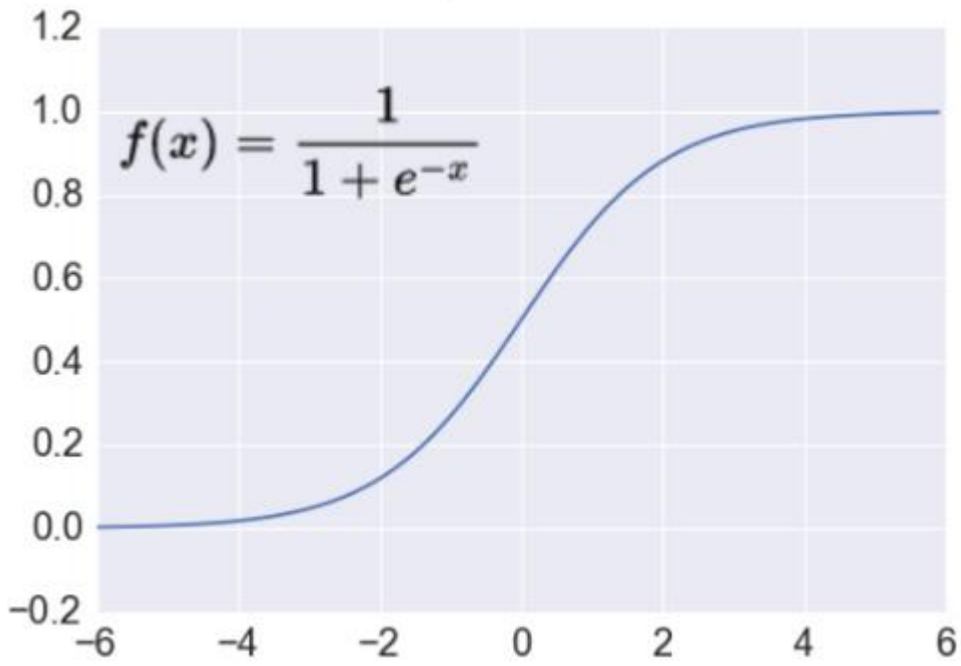
http://cs231n.github.io/assets/nn1/layer_sizes.jpeg

Full list: https://en.wikipedia.org/wiki/Activation_function

Activation Function: Sigmoid

Takes a real-valued number and "squashes" it into range between 0 and 1.

$$R^n \rightarrow [0,1]$$



+ Nice interpretation as the **firing rate** of a neuron
  - 0 = not firing at all
  - 1 = fully firing

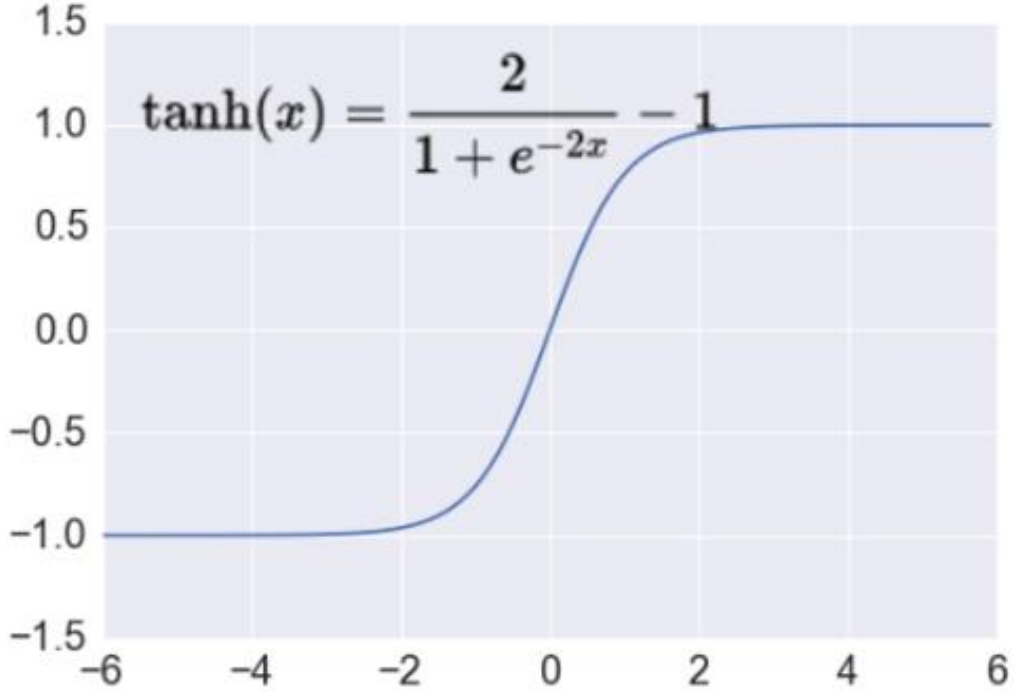
- Sigmoid neurons **saturate** and **kill gradients**, thus NN will barely learn
  - when the neuron's activation are 0 or 1 (saturate)
    - 🙁 gradient at these regions almost zero
    - 🙁 almost no signal will flow to its weights
    - 🙁 if initial weights are too large then most neurons would saturate

http://adilmoujahid.com/images/activation.png

Activation Function: Tanh

Takes a real-valued number and "squashes" it into range between -1 and 1.

$$R^n \rightarrow [-1,1]$$



- Like sigmoid, tanh neurons **saturate**
- Unlike sigmoid, output is **zero-centered**
- Tanh is a **scaled sigmoid**: $\tanh(x) = 2sigm(2x) - 1$
- Drawbacks of Sigmoid with Tanh also.

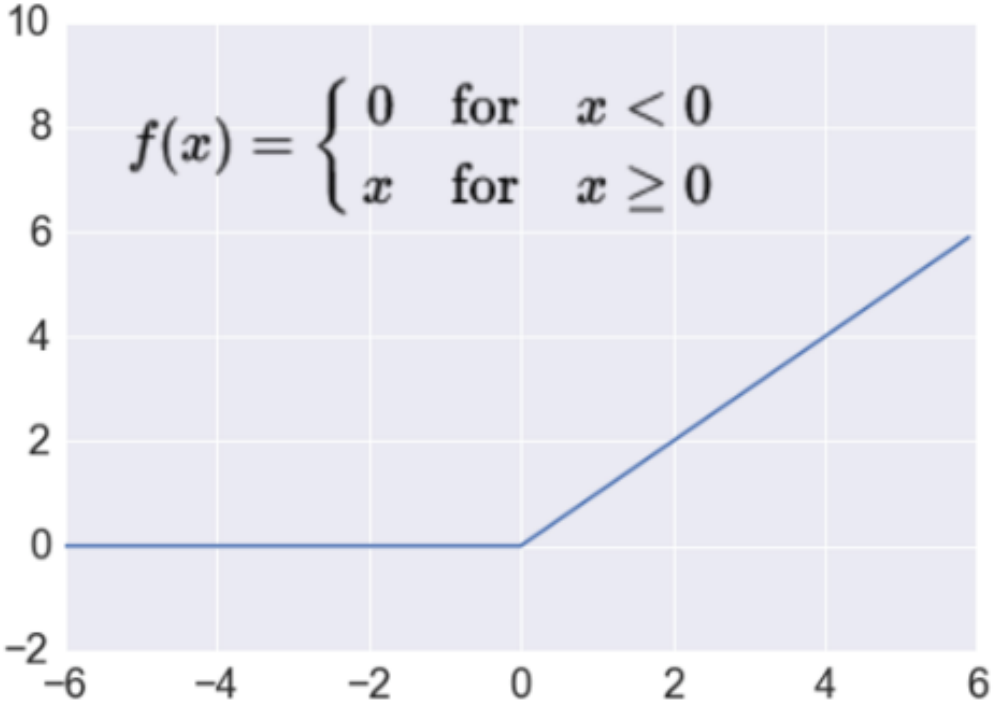
http://adilmoujahid.com/images/activation.png

Activation Function: ReLU

Takes a real-valued number and thresholds it at zero

$$f(x) = \max(0, x)$$

$$R^n \rightarrow R^n_+$$



Most Deep Networks use ReLU nowadays

🙂 Trains much **faster**

- accelerates the convergence of SGD
- due to linear, non-saturating form

🙂 Less expensive operations

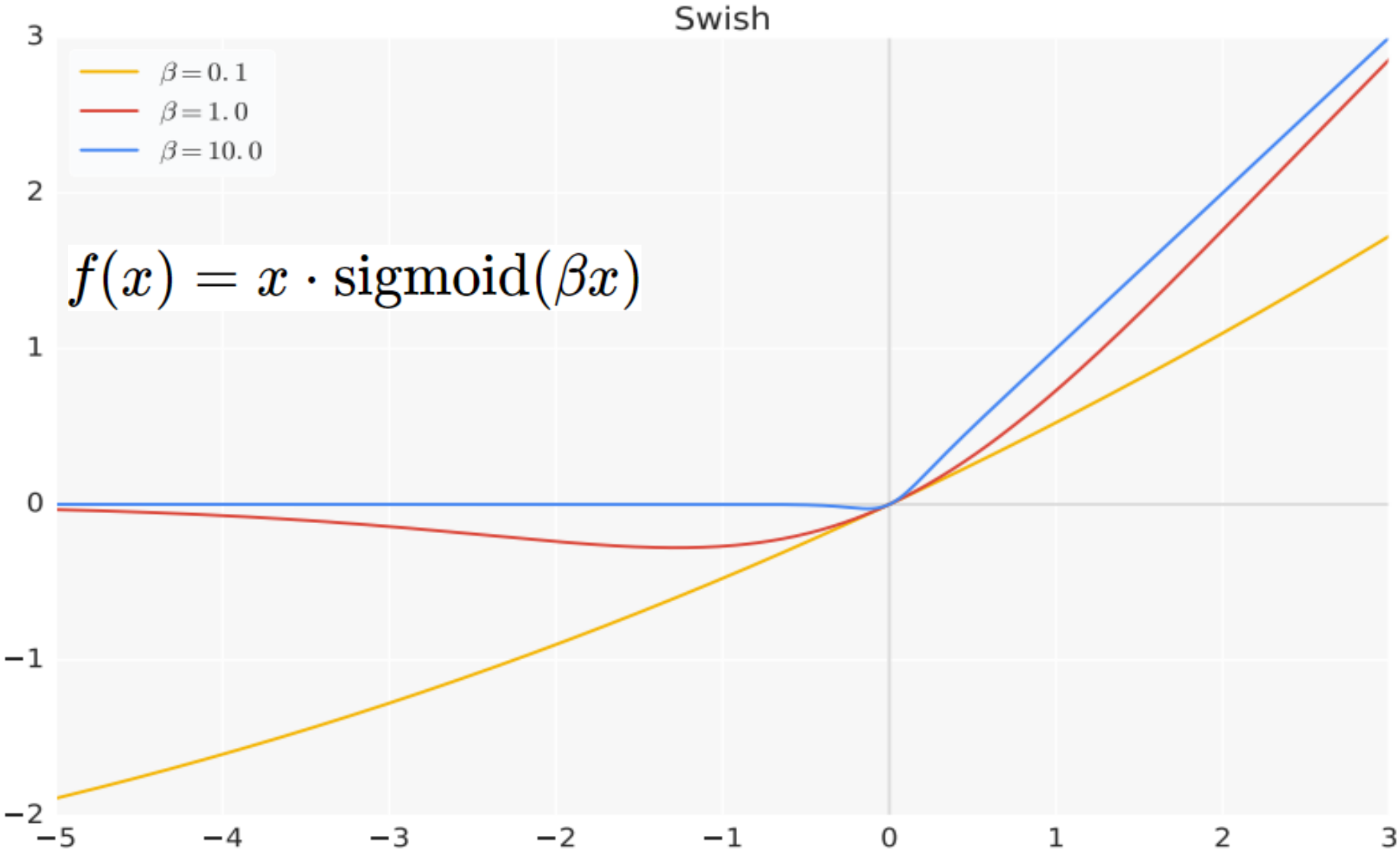
- compared to sigmoid/tanh (exponentials etc.)
- implemented by simply thresholding a matrix at zero

🙂 More **expressive**

🙂 Prevents the **gradient vanishing problem** for +ive inputs

http://adilmoujahid.com/images/activation.png

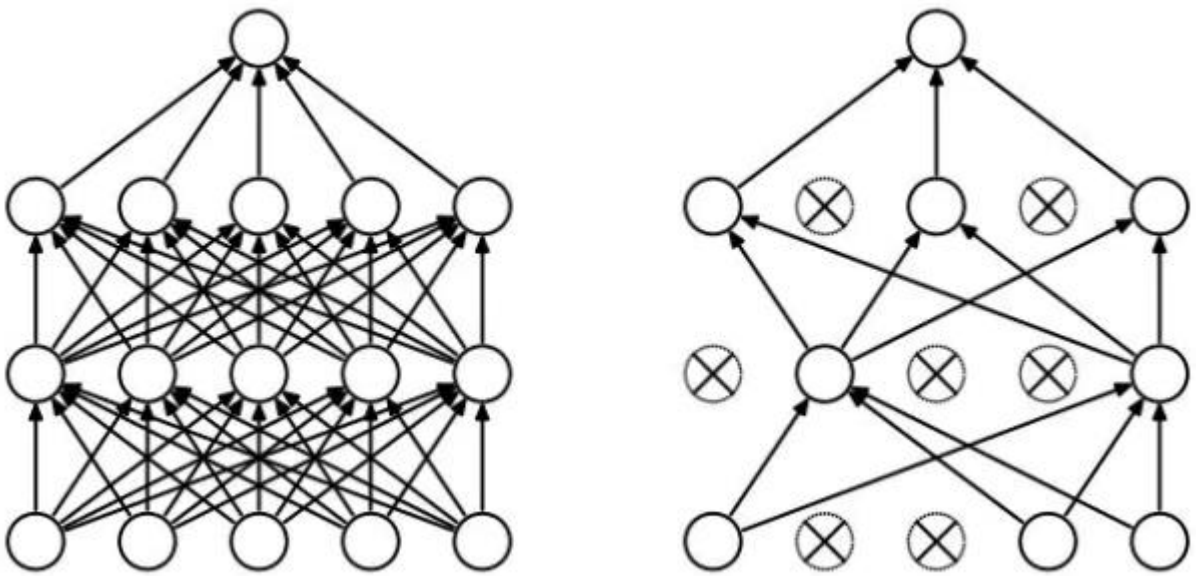
Activation Function: Swish



- ReLU is special case of Swish

Ramachandran et al. "Swish: a self-gated activation function." *ICLR Workshops*, 2018.

Regularization



Dropout

- Randomly drop units (along with their connections) during training
- Each unit retained with fixed probability p, independent of other units
- Hyper-parameter p to be chosen (tuned)

Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting." Journal of machine learning research (2014)

Batch Normalization

"We want zero-mean unit-variance activations? lets make them so."

consider a batch of activations at some layer. To make each dimension zero-mean unit-variance, apply:

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift [Ioffe and Szegedy 2015]

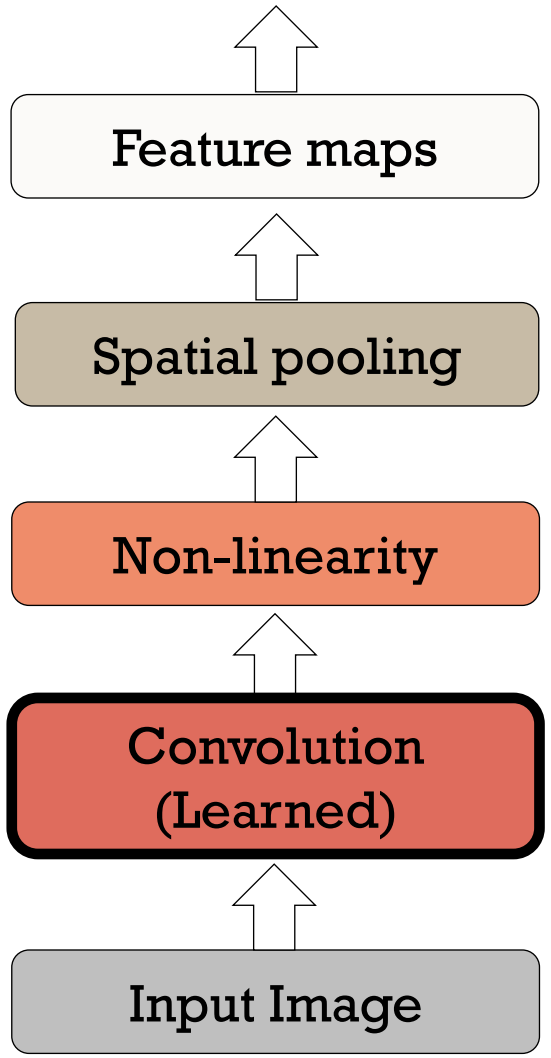
Source: cs231n

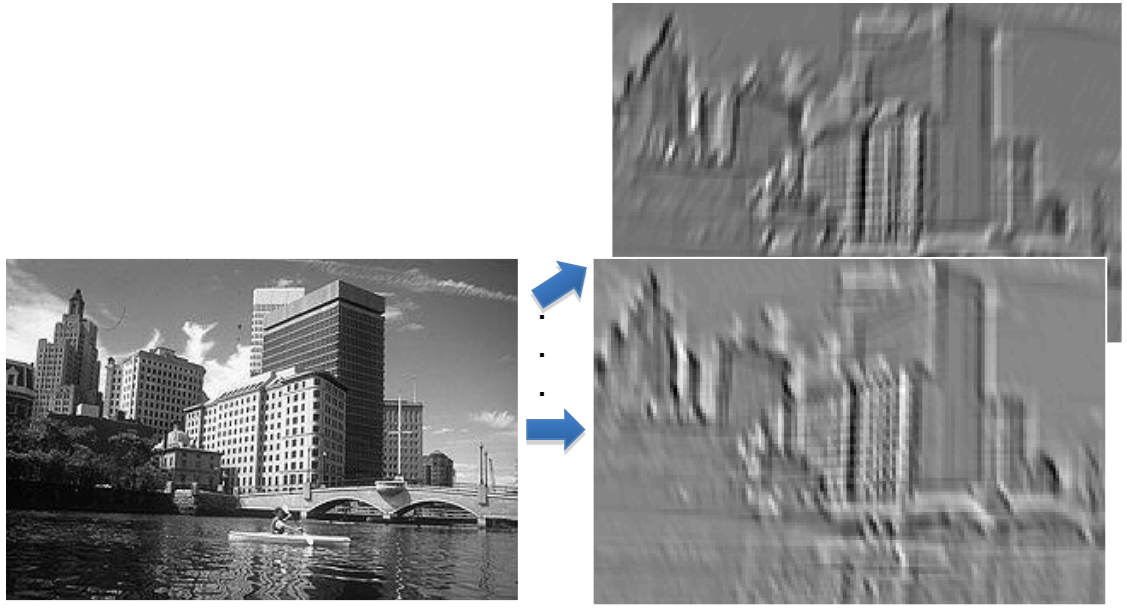
CLASSIFICATION/LOSS LAYER

- SVM Classifier
- SVM Loss/Hinge Loss/Max-margin Loss

- Softmax Classifier
- Softmax Loss/Cross-entropy Loss

SUMMARY: CNN PIPELINE



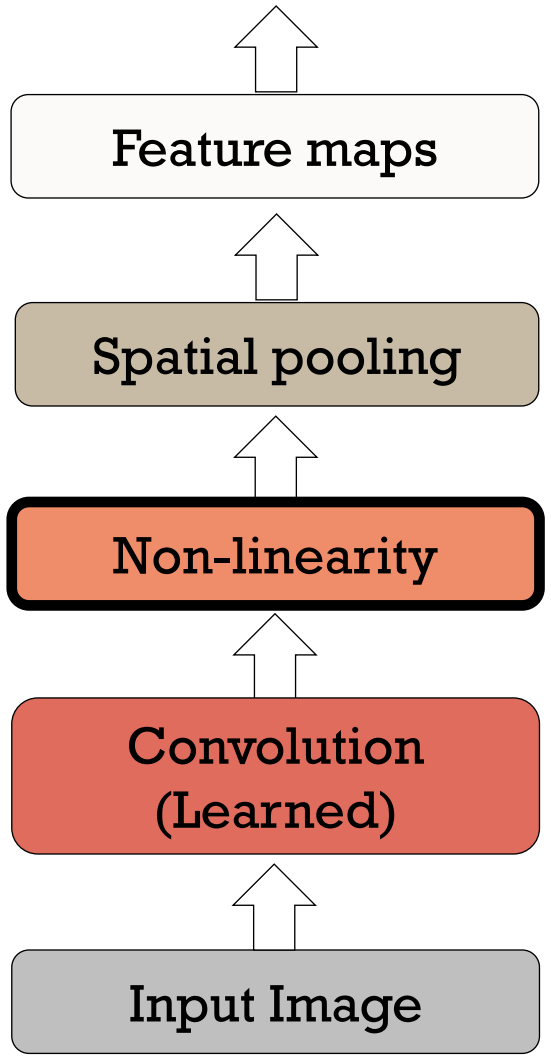


Input

Feature Map

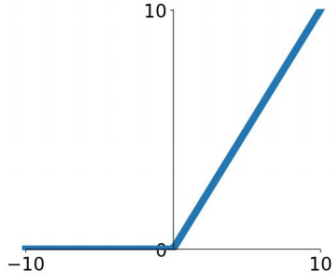
Source: R. Fergus, Y. LeCun

SUMMARY: CNN PIPELINE



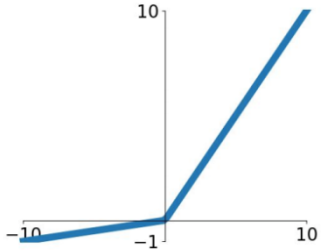
ReLU

$$\max(0, x)$$



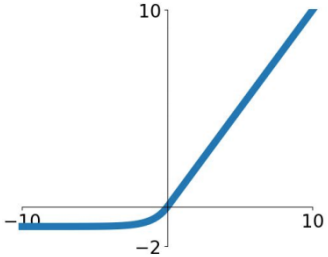
Leaky ReLU

$$\max(0.1x, x)$$



ELU

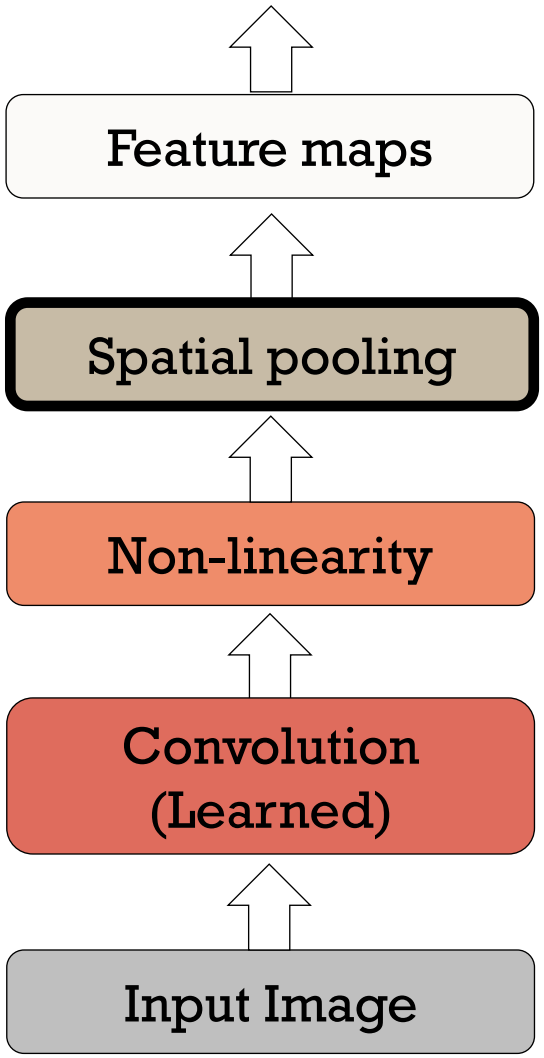
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

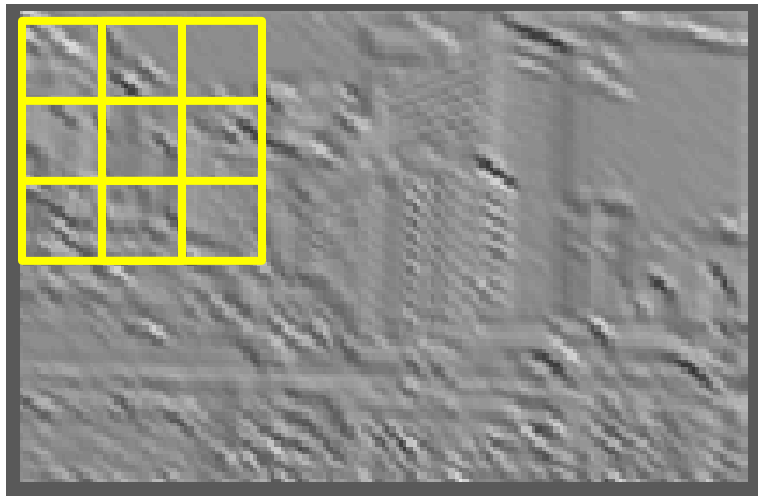


Source: R. Fergus, Y. LeCun

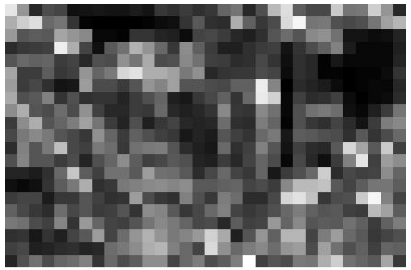
Source: Stanford 231n

SUMMARY: CNN PIPELINE



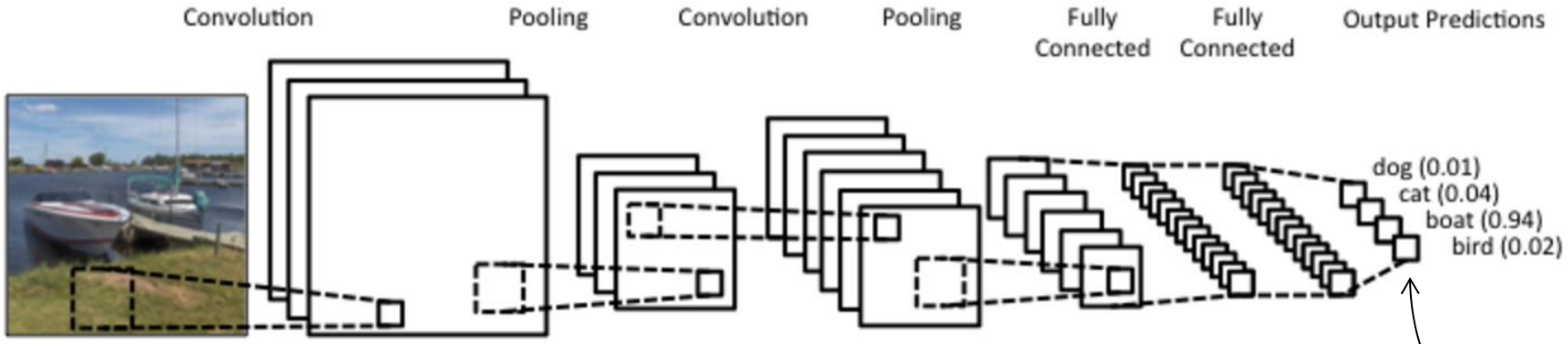


Max
(or Avg)



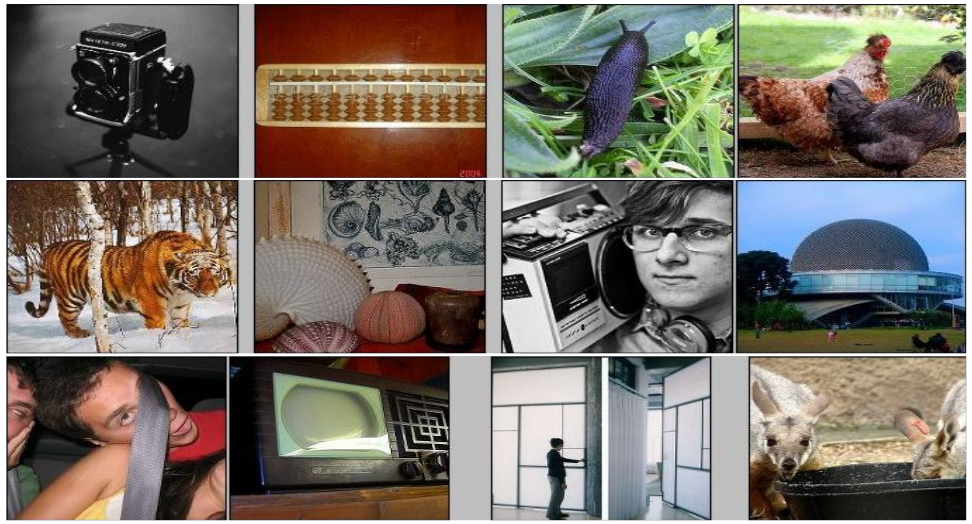
Source: R. Fergus, Y. LeCun

SUMMARY: CNN PIPELINE



Softmax layer:

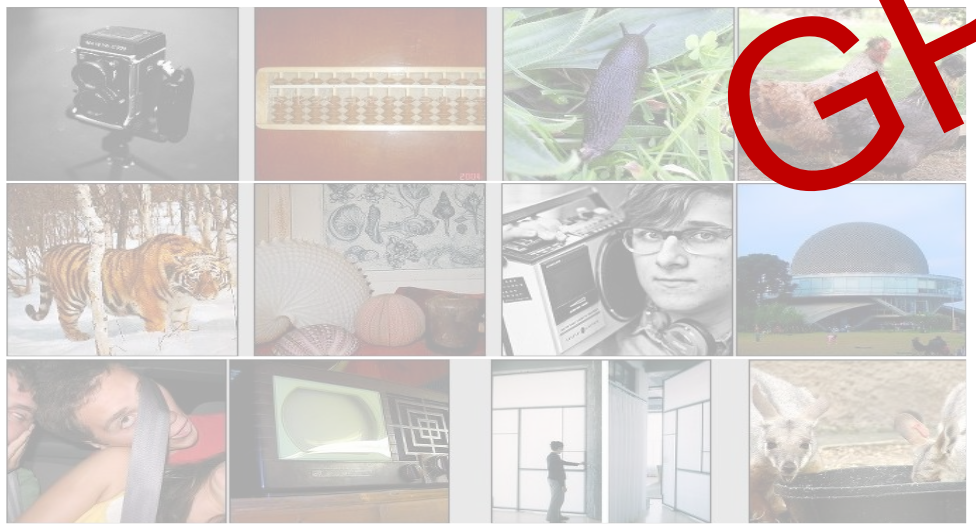
IMAGENET CHALLENGE



- ~14 million labeled images, 20k classes
- Images gathered from Internet
- Human labels via Amazon MTurk
- Challenge: 1.2 million training images, 1000 classes

www.image-net.org/challenges/LSVRC/

IMAGENET CHALLENGE

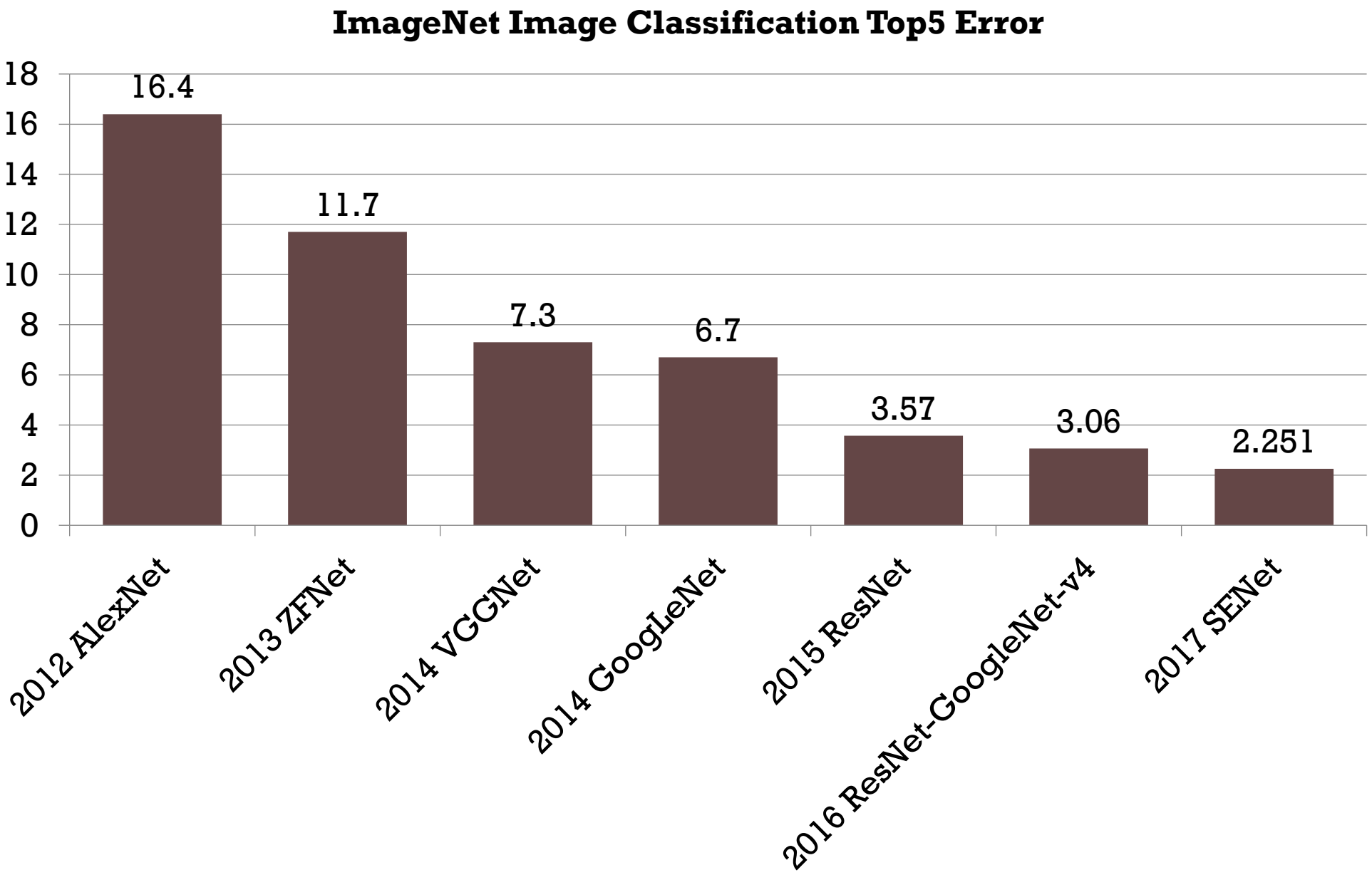


- 14 million labeled images, 20k classes
- Images gathered from Internet
- Human labels via Amazon MTurk
- Challenge: 1.2 million training images, 1000 classes

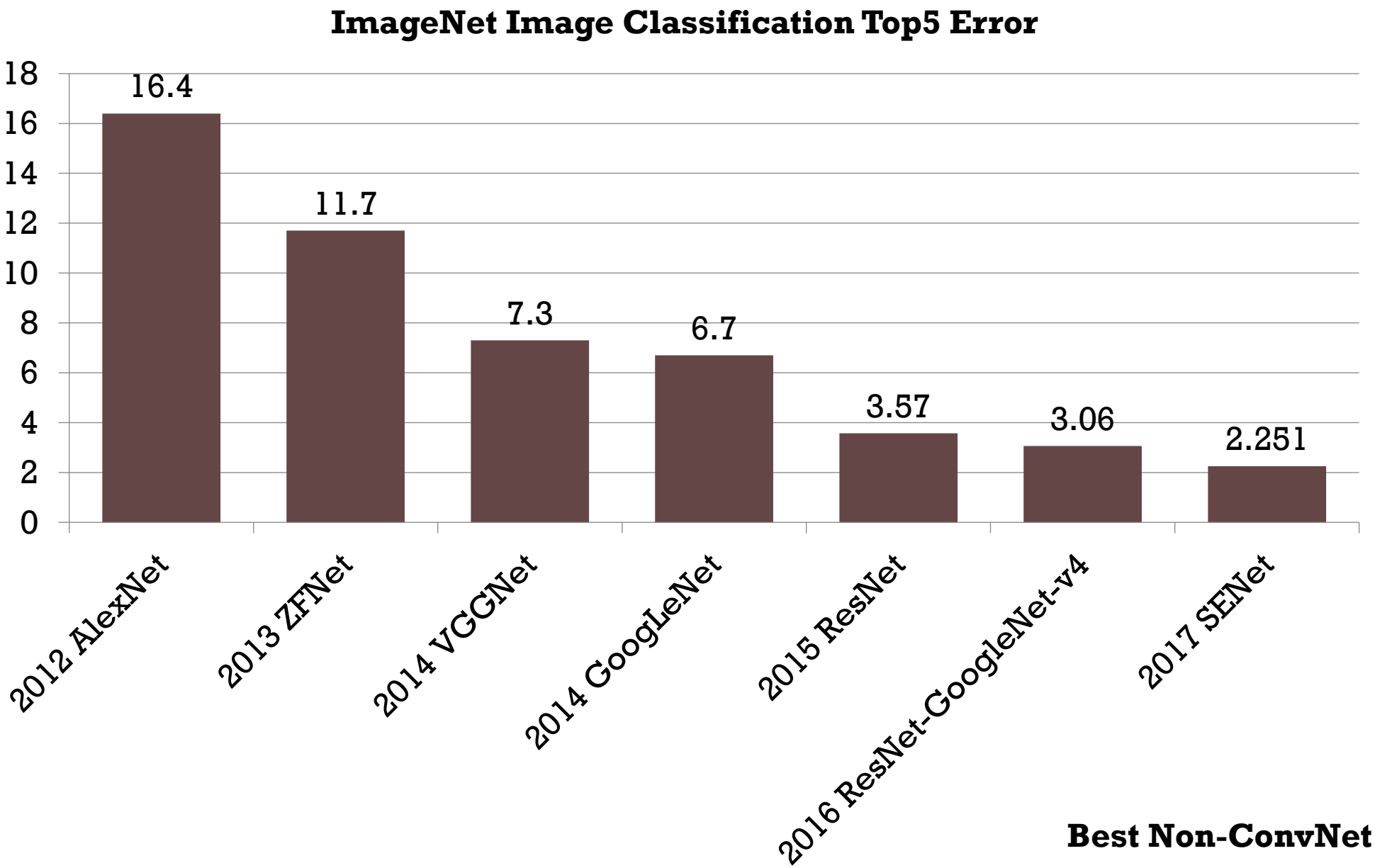
GPUs + Data

www.image-net.org/challenges/LSVRC/

PROGRESS ON IMAGENET CHALLENGE



PROGRESS ON IMAGENET CHALLENGE



Best Non-ConvNet in 2012: 26.2%

ACKNOWLEDGEMENT

Thanks to the following courses and corresponding researchers for making their teaching/research material online

- Convolutional Neural Networks for Visual Recognition, Stanford University
- Deep Learning, Stanford University
- Introduction to Deep Learning, University of Illinois at Urbana-Champaign
- Introduction to Deep Learning, Carnegie Mellon University
- Natural Language Processing with Deep Learning, Stanford University
- Ismini Lourentzou
- UIUC
- And Many More Publicly Available Resources .....

ABOUT IIIT ALLAHABAD (A RESEARCH LED INSTITUTE OF NATIONAL IMPORTANCE)

ABOUT COMPUTER VISION AND BIOMETRICS LAB (CVBL)

Funding

सत्यमेव जयते
Ministry of Education
Government of India

सत्यमेव जयते
Department of Science & Technology
Govt. of India

Global Innovation & Technology Alliance

NewGen IEDC
IIIT ALLAHABAD

Technically & Financially sponsored by
IEEE UP SECTION (INDIA)

IEEE Signal Processing Society

IEEE Computational Intelligence Society

IEEE youngprofessionals

Meerut Foundation

ABOUT COMPUTER VISION AND BIOMETRICS LAB (CVBL)

Ongoing Research Projects Funded By:

- I-DAPT HUB FOUNDATION, Indian Institute of Technology (BHU) Varanasi
  - CCTV Footage Super-resolution
- Naval Research Board, DRDO
  - Object Detection in Underwater Acoustic Images

Flagship Events:

- IEEE SPS Seasonal School on Explainable AI and Applications to Biometric Signal Processing, July 16-20, 2025
- IEEE CIS Summer School on Computational Advances in Deep Learning and Applications (CADLA), July 17-21, 2023
- IEEE International Conference on Computer Vision & Machine Intelligence (CVMI), 2022, 2024
- Short Term Course on Advances in Deep Architectures for Signal, Image & Vision Applications (ADASIVA), 2018, 2019, 2020, 2021, 2022
- Short Term Course Project on Advances In Biometric Signal Processing Techniques, 2021

IEEE SPS SEASONAL SCHOOL XAI @IIIT ALLAHABAD

https://cvbl.iiita.ac.in/seasonal-school/ieee-sps-xai2025/

THANK YOU

QUESTIONS?

Image Source: http://pluspng.com/best-of-luck-png-5072.html